

Результат ревью:

Красных комментариев, которые бы не позволяли мне принять твой проект в его нынешнем состоянии - нет.

ПРОЕКТ ПРИНЯТ!

Ввод []:

Ввод []:

КОММЕНТАРИЙ V6

1. (Не исправлено в последние 2 итерации) Каков порядок следования категорий в выбранных тобой ранговых признаках? Смотри в документации <https://scikit-learn.org/stable/modules/generated/sklearn.preprocessing.OrdinalEncoder.html> (<https://scikit-learn.org/stable/modules/generated/sklearn.preprocessing.OrdinalEncoder.html>) параметр `categories`.

```
# СОЗДАЕМ СЛОЖНЫЙ COLUMNTRANSFORMER С РАЗНЫМИ ТРАНСФОРМЕРАМИ
preprocessor = ColumnTransformer(
    transformers=[
        # Пайплайн для стабильных числовых признаков (StandardScaler)
        ('num_std', Pipeline(steps=[
            ('imputer', SimpleImputer(strategy='median')),
            ('scaler', StandardScaler())
        ]), numerical_std),

        # Пайплайн для числовых признаков с выбросами (RobustScaler)
        ('num_robust', Pipeline(steps=[
            ('imputer', SimpleImputer(strategy='median')),
            ('scaler', RobustScaler())
        ]), numerical_robust),

        # Пайплайн для ранговых категориальных признаков (OrdinalEncoder)
        ('cat_ordinal', Pipeline(steps=[
            ('imputer', SimpleImputer(strategy='most_frequent')),
            ('encoder', OrdinalEncoder(handle_unknown='use_encoded_value', unknown_value=np.nan)),
            ('simpleImputer_after_ord', SimpleImputer(missing_values=np.nan, strategy='most_frequent'))
        ]), categorical_ordinal),

        # Пайплайн для номинальных категориальных признаков (OneHotEncoder)
        ('cat_oh', Pipeline(steps=[
            ('imputer', SimpleImputer(strategy='most_frequent')),
            ('encoder', OneHotEncoder(handle_unknown='ignore', sparse=False))
        ]), categorical_oh)
    ],
    remainder='drop'
)
```

Добавил порядок следования.

КОММЕНТАРИЙ V7

Да, вижу, подтверждаю исправление. ПРОЕКТ ПРИНЯТ.

КОММЕНТАРИЙ V5

1. (Не исправлено с прошлой итерации) Каков порядок следования категорий в выбранных тобой ранговых признаках? Смотри в документации <https://scikit-learn.org/stable/modules/generated/sklearn.preprocessing.OrdinalEncoder.html> (<https://scikit-learn.org/stable/modules/generated/sklearn.preprocessing.OrdinalEncoder.html>) параметр `categories`.

```
# Пайплайн для ранговых категориальных признаков (OrdinalEncoder)
('cat_ordinal', Pipeline(steps=[
    ('imputer', SimpleImputer(strategy='most_frequent')),
    ('encoder', OrdinalEncoder(handle_unknown='use_encoded_value', unknown_value=np.nan)),
    ('simpleImputer_after_ord', SimpleImputer(missing_values=np.nan, strategy='most_frequent'))
]), categorical_ordinal),
```

2. 'Популярную_категорию' разве можно отнести к ранговым/порядковым/упорядоченным признакам? 'Товары для детей' лучше 'Домашний текстиля' или 'Домашний текстиль' лучше 'Товаров для детей'? А 'Кухонная посуда' лучше или хуже 'Товаров для детей'?

'Популярная_категория' - это скорее номинальный признак.

r.s. номинальный признак - это тот, значения которой представляют категории без естественного упорядочения: например, подразделение компании, где работает наёмный сотрудник. Примеры номинальных переменных включают название региона, цвет одежды.

```
# Признаки, которые исключаем из анализа
excluded_features = ['id', 'страница_за_визит', 'время_пред', 'маркет_актив_тек_мес', 'выручка_пред']

# Определяем все возможные признаки
categorical_features = ['тип_сервиса', 'разрешить_сообщать', 'популярная_категория']
numerical_features = [
    'маркет_актив_б_мес', 'длительность', 'акционные_покупки',
    'средний_просмотр_категорий_за_визит', 'неоплаченные_продукты_штук_квартал',
    'ошибка_сервиса', 'выручка_тек', 'время_текущий_месяц'
]

# Проверяем, какие признаки действительно есть в DataFrame
existing_columns = final_df.columns.tolist()
available_categorical = [f for f in categorical_features if f in existing_columns and f not in excluded_features]
available_numerical = [f for f in numerical_features if f in existing_columns and f not in excluded_features]
```

Поставил на орд 'тип_сервиса', 'разрешить_сообщать'

КОММЕНТАРИЙ V6

Да, этот момент вижу, исправлено.

КОММЕНТАРИЙ V4

1. (32) Стоит настроить работу энкодера `OrdinalEncoder` на работу с неизвестными значениями признаков - сейчас энкодер "при встрече с неизвестной категорией" (это может случиться в проде) - будет выбрасывать ошибку. Для непрерывности процесса моделирования можно настроить параметры `handle_unknown='use_encoded_value'`, чтобы новые, ранее неизвестные значения кодировались константным значением, а в параметр `unknown_value` нужно передать само значение-константу. Если же замена неизвестных значений будет выполняться на `np.nan`, то после `OrdinalEncoder()` нужен будет `SimpleImputer()`.
2. (32) Каков порядок следования категорий в выбранных тобой ранговых признаках? Смотри в документации <https://scikit-learn.org/stable/modules/generated/sklearn.preprocessing.OrdinalEncoder.html> (<https://scikit-learn.org/stable/modules/generated/sklearn.preprocessing.OrdinalEncoder.html>) параметр **categories**.

```
# Пайплайн для ранговых категориальных признаков (OrdinalEncoder)
('cat_ordinal', Pipeline(steps=[
    ('imputer', SimpleImputer(strategy='most_frequent')),
    ('encoder', OrdinalEncoder())
]), categorical_ordinal),
```

3. (32) Код упал с ошибкой:

```
test_roc_auc = roc_auc_score(y_test, y_pred_proba)
print(f"ROC-AUC простой модели: {test_roc_auc:.4f}")
```

Доступные категориальные признаки: ['тип_сервиса', 'разрешить_сообщать', 'популярная_категория']
Доступные числовые признаки: ['маркет_актив_6_мес', 'длительность', 'акционные_покупки', 'неоплаченные_продукты_штук_квартал', 'ошибка_сервиса', 'выручка_тек', 'время_текущий_месяц']

Группы для трансформеров:

Numerical (StandardScaler): ['маркет_актив_6_мес', 'длительность', 'акционные_покупки', 'ошибка_сервиса', 'время_текущий_месяц']

Numerical (RobustScaler): ['неоплаченные_продукты_штук_квартал', 'выручка_тек']

Categorical (OrdinalEncoder): ['тип_сервиса']

Categorical (OneHotEncoder): ['разрешить_сообщать', 'популярная_категория']

```
-----
TypeError                                Traceback (most recent call last)
/tmp/ipykernel_31/1330838479.py in <module>
     86         ('cat_ohe', Pipeline(steps=[
     87             ('imputer', SimpleImputer(strategy='most_frequent')),
--> 88             ('encoder', OneHotEncoder(handle_unknown='ignore', sparse_out
     89         ])), categorical_ohe)
     90     ],
```

```
/opt/conda/lib/python3.9/site-packages/sklearn/utils/validation.py in inner_f(*args, **kwargs)
     61     extra_args = len(args) - len(all_args)
     62     if extra_args <= 0:
--> 63         return f(*args, **kwargs)
     64
     65     # extra_args > 0
```

```
TypeError: __init__() got an unexpected keyword argument 'sparse_output'
```

OrdinalEncoder настроил, значения для него изменил на

Ввод []:

КОММЕНТАРИЙ V3

Необходимо исправить ✖

```
# СОЗДАЕМ СЛОЖНЫЙ COLUMNTRANSFORMER С РАЗНЫМИ ТРАНСФОРМЕРАМИ
preprocessor = ColumnTransformer(
    transformers=[
        # Пайплайн для стабильных числовых признаков (StandardScaler)
        ('num_std', Pipeline(steps=[
            ('imputer', SimpleImputer(strategy='median')),
            ('scaler', StandardScaler())
        ]), numerical_std),

        # Пайплайн для числовых признаков с выбросами (RobustScaler)
        ('num_robust', Pipeline(steps=[
            ('imputer', SimpleImputer(strategy='median')),
            ('scaler', RobustScaler()) # Другой скейлер!
        ]), numerical_robust),

        # Пайплайн для категориальных признаков (OneHotEncoder)
        ('cat_oh', Pipeline(steps=[
            ('imputer', SimpleImputer(strategy='most_frequent')),
            ('encoder', OneHotEncoder(handle_unknown='ignore', sparse=False))
        ]), categorical_oh)
    ],
    remainder='drop' # Игнорируем все остальные признаки
)

# СОЗДАЕМ ОСНОВНОЙ ПАЙПЛАЙН
pipe_final = Pipeline(steps=[
    ('preprocessor', preprocessor),
    ('classifier', None) # Будем подбирать
])
```

2 скейлера вижу, но 2го кодировщика (энкодера) НЕ вижу.
В наших данных есть один ранговый признак (уровень сервиса) подходящий для кодирования его OrdinalEncoder'ом (это подсказка).

Не исправлено.

Ввод []:

Ввод []:

КОММЕНТАРИЙ V2

Итак, мои комментарии на этот раз будут зелёными, в заголовке иметь версию V2 (как у этого!). Комментарии будут зелеными, даже если в них будут описаны критические ошибки.

Здесь, ниже, по пунктам я эти ошибки перечислю, если таковые будут обнаружены.

Я мелкие/"проходные" изменения/исправления в тексте проекта, каждое из них, не буду с твоего позволения комментировать - но только те моменты, где я что-то более-менее не банальное могу сказать. Хорошо? Будем считать, что, если комментария на твоё исправление нет - значит я согласен с тобой в этом пункте.

Итак, на этой итерации всё-таки остались моменты с прошлого ревью, которые тебе надо исправить. Перечислю их:

1. (20) Ряд признаков в наших данных имеет дискретную природу (например это: 'Маркет_актив_тек_мес', 'Средний_просмотр_категорий_за_визит', ... (не привожу все названия из данных)). Для графического анализа дискретных признаков нужен тип графиков, который на каждое уникальное значение признака строит свою корзину. Например, можно использовать `sns.countplot` или `.plot(kind='bar')`. Ты же для визуализации их используешь гистограмму.
2. (26) Все ли признаки в `interval_cols` сейчас действительно непрерывные? Или "затесались" в список ещё и дискретные?

3. (30) Артемий - не вижу исправления. Вижу только один кодировщик и один скейлер. ДОЛЖНО БЫТЬ: в ColumnTransformer видно что используется 2 скейдера и 2 кодировщика, причём кодировщики должны кодировать каждый - собственный набор признаков.

Жду твою работу на повторное ревью.

Ввод []:

Ввод []:

Вновь приветствую тебя Артемий!

Меня зовут Александр Непомнящий и я буду проверять твой проект.

Вкратце обо мне: я работаю в сфере информационных технологий, аналитика (анализ данных - это и мой рабочий функционал и личная, интересная мне область для саморазвития. Используемый стек: python, pandas, графические библиотеки matplotlib/seaborn/plotly. Сфера применения: маркетинговая аналитика в медицинской клинике. Если вдруг есть вопрос – что делает аналитик в коммерческой медицине – то вот короткий ответ. Самый главный вопрос, на который мы хотим получить ответ (и который в дальнейшем приведёт к перераспределению рекламного бюджета, плюс напрямую повлияет на разработку и предложению рынку новых лечебных программ) - это КТО ОН, НАШ КЛИЕНТ? Фактически это задача (на нашем внутреннем языке формулируется как построение и анализ «ПОРТРЕТА КЛИЕНТА»). Визуализируем и пытаемся придать смысл половозрастным данным наших клиентов, кто из какого региона и в какие времена года к нам приезжает. Учитываем количество приездов. Учитываем на какую программу этот конкретный клиент приехал в первый раз и как в дальнейшем меняются его предпочтения. Пытаемся построить модель оттока и спрогнозировать, когда клиент уйдёт в отток. Смотрим цифры (суммы трат клиентов на отдельных группах медицинских программ). Анализируем медицинские услуги, которые входят в путёвку, а также что, в каких количествах и на какую сумму клиент покупает дополнительно. Очень вкратце как-то так.

Теперь к сути: основная цель моей проверки - пройти по шагам, по логике твоего решения и обсудить те моменты, которые можно изменить, улучшить, довести до идеала. Такой формат работы двух программистов (аналитиков) ты или уже встречал(а), или встретишь на своей новой работе. Кто-то такой процесс называет "код-ревью", ну а как по мне, так это обмен опытом двух коллег, когда каждый из нас становится лучше в процессе дискуссии.

Свои мысли, советы, комментарии и вопросы я буду оставлять в текстовых ячейках (markdown) ПОД твоим кодом. Для твоего удобства все эти заметки будут "цветными", как на примерах ниже:

"Зелеными" комментариями я буду отмечать твой элегантный ("красивый") код и/или удачные решения, на которые ты можешь смело опираться в будущих проектах.

Отлично! 👍

Отличная находка, элегантный код ...

"Жёлтые" комментарии - это мои размышления, предложения, что-то, что на следующей итерации можно сделать по-другому. Твоя реакция на "жёлтые" комментарий необходима и подразумевает или правки, или реакцию или аргументированное согласие/не согласие. Я искренне верю, что наш двусторонний диалог под "желтым" комментарием - это то, от чего твой проект только выиграет.

Комментарий 👉

Здесь я бы предложил использовать Мои аргументы в пользу этого ...

"Красные" комментарий будут появляться, когда что-то пойдёт не так: например, код будет выдавать ошибку. Или, когда в проекте не будет ответов на основные вопросы, ради которых весь анализ и выполнялся. "Красные" комментарии необходимо будет исправить, прежде чем я смогу принять проект.

Необходимо исправить ❌

В этой ячейке код работает не так как задумано ...

Пожалуйста, не перемещай, не изменяй и не удаляй мои комментарии. Это поможет мне оперативно выполнить повторную проверку твоего проекта.

Давай работать над проектом в диалоге: если ты что-то меняешь в проекте по моим рекомендациям — пиши об этом, лучше всего в новой ячейке, сразу под моим комментарием. Если для своих комментариев ты будешь использовать какой-то заметный цвет, то мне будет легче отследить твои изменения, например так:

КОММЕНТАРИЙ СТУДЕНТА

Текст твоего комментария ...

Распространенные типичные недочёты в проектах студентов:

Неработоспособный код. Проверь проект на работоспособность перед отправкой на проверку. Это важный навык, который тебе пригодится как на собеседовании, при выполнении тестового задания, так и перед совещанием у руководства компании.

Проект не структурирован. В проекте должна быть структура, т.е. названия разделов и подразделов должны быть записаны в ячейках Markdown.

Отсутствие описания, целей проекта, его задач, выводов. В описании должны быть озвучены поставленные задачи, содержание датасета, промежуточные и финальные выводы. Как правило, заказчик читает только финальный вывод, поэтому важно добавить информацию о проведенном исследовании, применить форматирование текста, в общем, сделать красиво, чтобы было понятно, что проведено настоящее исследование и выводу можно верить.

"Длинный код". Код и комментарии, которые прячутся за правым концом ячейки нужно переносить на новую строку. Твои будущие коллеги не оценят необходимость постоянного скроллинга. Согласно PEP8: <https://www.python.org/dev/peps/pep-0008/> длина строки не должна превышать 79 символов.

Твои коллеги ценят и твое и своё время, поэтому "подсвечивай" свои комментарии в ячейках Markdown. Подробнее про синтаксис Markdown, с

Александр, снова привет!) Рад, что ты проверяешь проект. Многие из твоего предыдущего ревью подчеркнул для себя.

КОММЕНТАРИЙ V2

Артемию привет! Отлично, рад помогать коллеге! Как обычно, сейчас смотрю исправления и если всё ОК, то проект незамедлительно будет принят!

Ввод []:

Ввод []:

Проект: Обучение с учителем: качество модели

Цель проекта:

- Разработать решение, которое позволит персонализировать предложения постоянным клиентам, чтобы увеличить их покупательскую активность.

План проекта

- Разработать модель, которая предскажет вероятность снижения покупательской активности.
- Выделить сегмент покупателей, проанализировать его и предложить, как увеличить его покупательскую активность.

-
- Шаг 1. Загрузка данных
 - Шаг 2. Предобработка данных
 - Шаг 3. Исследовательский анализ данных
 - Шаг 4. Объединение таблиц
 - Шаг 5. Корреляционный анализ
 - Шаг 6. Использование пайплайнов
 - Шаг 7. Анализ важности признаков

- Шаг 8. Сегментация покупателей
- Шаг 9. Общий вывод

Описание данных

- **market_file.csv** (Таблица, которая содержит данные о поведении покупателя на сайте, о коммуникациях с покупателем и его продуктовом поведении.)
 - id — номер покупателя в корпоративной базе данных.
 - Покупательская активность — рассчитанный класс покупательской активности (целевой признак): «снизилась» или «прежний уровень».
 - Тип сервиса — уровень сервиса, например «премиум» и «стандарт». Зависит от наличия определенной подписки у пользователя.
 - Разрешить сообщать — информация о том, можно ли присылать покупателю дополнительные предложения о товаре. Согласие на это даёт покупатель.
 - Маркет_активбмес — среднеемесячное значение маркетинговых коммуникаций компании, которое приходилось на покупателя за последние 6 месяцев. Это значение показывает, какое число рассылок, звонков, показов рекламы и прочего приходилось на клиента.
 - Маркет_актив_тек_мес — количество маркетинговых коммуникаций в текущем месяце.
 - Длительность — значение, которое показывает, сколько дней прошло с момента регистрации покупателя на сайте.
 - Акционные_покупки — среднеемесячная доля покупок по акции от общего числа покупок за последние 6 месяцев.
 - Популярная_категория — самая популярная категория товаров у покупателя за последние 6 месяцев.
 - Средний_просмотр_категорий_за_визит — показывает, сколько в среднем категорий покупатель просмотрел за визит в течение последнего месяца.
 - Неоплаченные_продукты_штук_квартал — общее число неоплаченных товаров в корзине за последние 3 месяца.
 - Ошибка_сервиса — число сбоев, которые коснулись покупателя во время посещения сайта за последние 3 месяца.
 - Страниц_за_визит — среднее количество страниц, которые просмотрел покупатель за один визит на сайт за последние 3 месяца.

- **market_money.csv** (Таблица с данными о выручке, которую получает магазин с покупателя, то есть сколько покупатель всего потратил за период взаимодействия с сайтом.)
 - id — номер покупателя в корпоративной базе данных.
 - Период — название периода, во время которого зафиксирована выручка. Например, 'текущий_месяц' или 'предыдущий_месяц'.
 - Выручка — сумма выручки за период в условных единицах.

- **market_time.csv** (Таблица с данными о времени (в минутах), которое покупатель провёл на сайте в течение периода.)
 - id — номер покупателя в корпоративной базе данных.
 - Период — название периода, во время которого зафиксировано общее время.
 - минут — значение времени, проведённого на сайте, в минутах.

- **money.csv** (Таблица с данными о среднемесячной прибыли продавца за последние 3 месяца: какую прибыль получает магазин от продаж каждому покупателю.)
 - id — номер покупателя в корпоративной базе данных.
 - Прибыль — значение прибыли в условных единицах.

ОТЛИЧНО! 👍

Хорошее, качественное Введение!

1. Загрузка данных

Загрузим необходимые библиотеки.

```
Ввод [1]: import pandas as pd
import re
import matplotlib.pyplot as plt
import seaborn as sns
import numpy as np
from matplotlib.ticker import MultipleLocator
from statsmodels.stats.outliers_influence import variance_inflation_factor
from sklearn.pipeline import Pipeline
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import OneHotEncoder, OrdinalEncoder, StandardScaler
from sklearn.impute import SimpleImputer
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from scipy.stats import uniform, randint
from sklearn.neighbors import KNeighborsClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.svm import SVC
from sklearn.model_selection import RandomizedSearchCV
from sklearn.cluster import KMeans
from sklearn.inspection import permutation_importance
from sklearn.preprocessing import LabelEncoder
from sklearn.ensemble import RandomForestClassifier
from sklearn.preprocessing import RobustScaler
from sklearn.metrics import roc_auc_score, accuracy_score, f1_score, confusion_matrix
from sklearn.dummy import DummyClassifier
```

```
Ввод [2]: pip install shap -q
```

Note: you may need to restart the kernel to use updated packages.

```
Ввод [3]: !pip install phik -q
import phik
```

ОТЛИЧНО! 👍

Ввод [4]: **import** shap

ОТЛИЧНО! 👍

Отлично, все импорты собраны в начале "тетрадки"!

В Python есть своя философия. Именно отсюда берутся определенные правила и стандарты. В пайтон такое соглашение называли PEP8.

Соответствие PEP8 позволяет видеть основную структуру вашего кода другим разработчикам. Не нужно привыкать к другому оформлению кода. Если вы привыкли как-то оформлять код, а увидели что-то совершенно другое, то вы поймете.

Официальная документация: <https://peps.python.org/pep-0008/>
(<https://peps.python.org/pep-0008/>).

Основные моменты и рекомендации:

- Используйте пробелы, разрешается использование табов, но смешивание запрещено. Отступ соответствует 4 пробелам. Хорошие редакторы кода автоматически заменяют табы на пробелы.
- Вокруг операторов +, -, =, , /, * должны быть пробелы.
- Строка не должна превышать 79 символов
- Ну и наконец про импорты: импорты можно делать только по одному на строку! При этом из одного модуля можно доставать несколько нужных объектов. Порядок тоже важен:
 - импорты из стандартной библиотеки
 - импорты из сторонних библиотек
 - импорты пользовательских модулей текущего проекта (например если выносишь функции в отдельный модуль)

И нежелательно их перемешивать с константами и переменными.

p.s. состав стандартных библиотек Python здесь: [The Python Standard Library](https://docs.python.org/3/library/)
(<https://docs.python.org/3/library/>).

Далее загружаем и смотрим все датафреймы.

Ввод [5]: `market_file = pd.read_csv('/datasets/market_file.csv')`
`market_money = pd.read_csv('/datasets/market_money.csv')`
`market_time = pd.read_csv('/datasets/market_time.csv')`
`money = pd.read_csv('/datasets/money.csv', sep=';')`

ОТЛИЧНО! 👍

Отлично. Импортировали требуемые для анализа библиотеки и загрузили данные. Сейчас посмотрим на их состояние и если всё ОК, сможем приступить к построению моделей....

p.s. при установке дополнительных библиотек (через ! pip install ...) можно использовать полезный ключ `-q` (он "подавит" служебные сообщения об установке, которых может быть достаточно много - вплоть до нескольких экранов прокрутки)

p.p.s. При считывании данных из файла здорово перестраховывать себя от ошибок, связанных, например, с неверным указанием пути к файлу. А иногда бывает, что работаешь с файлом локально, выгружаешь его на сервер, ожидая, что он будет принимать данные, которые лежат на том же сервере, а код падает с ошибкой, потому что путь к файлу не поменялся с локального на серверный.

Для этого, например, можно использовать конструкцию `try-except` : сначала пробуешь локальный путь, при возникновении ошибки используется серверный путь (подробнее можешь почитать тут: <https://pythonworld.ru/typy-dannyx-v-python/isklyucheniya-v-python-konstrukciya-try-except-dlya-obrabotki-isklyuchenij.html> (<https://pythonworld.ru/typy-dannyx-v-python/isklyucheniya-v-python-konstrukciya-try-except-dlya-obrabotki-isklyuchenij.html>)).

Но еще лучше использовать библиотеку `os` - её использование позволит тебе проверять существование указанных директорий (что может быть актуально при одновременной работе на локальном и сетевом окружении) и загружать данные из существующей директории, избегая ошибок. Как пример:

```
import os

pth1 = '/folder_1/data.csv'
pth2 = '/folder_2/data.csv'

if os.path.exists(pth1):
    query_1 = pd.read_csv(pth1)
elif os.path.exists(pth2):
    query_1 = pd.read_csv(pth2)
else:
    print('Something is wrong')
```

Ещё на этапе считывания данных можно спарсить дату: за это действие отвечает параметр `parse_dates` метода `read_csv()` , в него нужно передать список с названием полей-дат, и в большинстве случаев дата будет корректно преобразована в нужный формат сразу.) Также на этапе считывания данных

Ввод [6]:

```
display(market_file.head())
display(market_money.head())
display(market_time.head())
display(money.head())
```

	id	Покупательская активность	Тип сервиса	Разрешить сообщать	Маркет_актив_6_мес	Маркет_актив_тек_
0	215348	Снизилась	премиум	да	3.4	
1	215349	Снизилась	премиум	да	4.4	
2	215350	Снизилась	стандартт	нет	4.9	
3	215351	Снизилась	стандартт	да	3.2	
4	215352	Снизилась	стандартт	нет	5.1	

	id	Период	Выручка
0	215348	препредыдущий_месяц	0.0
1	215348	текущий_месяц	3293.1
2	215348	предыдущий_месяц	0.0
3	215349	препредыдущий_месяц	4472.0
4	215349	текущий_месяц	4971.6

	id	Период	минут
0	215348	текущий_месяц	14
1	215348	предыдцщий_месяц	13
2	215349	текущий_месяц	10
3	215349	предыдцщий_месяц	12
4	215350	текущий_месяц	13

	id	Прибыль
0	215348	0,98
1	215349	4,16
2	215350	3,13
3	215351	4,87
4	215352	4,21

Данные не плохие, но нужны корректировки.

ОТЛИЧНО! 👍

Хорошо, данные загрузили, первичные проверки сделали.

2. Предобработка данных

Приведем все названия столбцов в "змеинный" регистр.

```
Ввод [7]: tables = [  
    ('market_file', market_file),  
    ('market_money', market_money),  
    ('market_time', market_time),  
    ('money', money)  
]  
  
for name, df in tables:  
    df.columns = [re.sub(r'(?                  for i in df.columns]  
    for col in df.columns:  
        if pd.api.types.is_string_dtype(df[col]):  
            df[col] = df[col].str.lower()  
  
for name, df in tables:  
    print(f"\n{name}")  
    print(df.columns.to_list())
```

```
market_file  
['id', 'покупательская_активность', 'тип_сервиса', 'разрешить_сообща  
ть', 'маркет_актив_6_мес', 'маркет_актив_тек_мес', 'длительность',  
'акционные_покупки', 'популярная_категория', 'средний_просмотр_катег  
орий_за_визит', 'неоплаченные_продукты_штук_квартал', 'ошибка_сервис  
а', 'страниц_за_визит']
```

```
market_money  
['id', 'период', 'выручка']
```

```
market_time  
['id', 'период', 'минут']
```

```
money  
['id', 'прибыль']
```

ОТЛИЧНО! 👍

Молодец, что названия столбцов выше приведены к "змеинному регистру"! р.с и приведение названий столбцов к нижнему регистру и изменение типов данных столбцов - это отдельные шаги в общем пайплайне исследовательского анализа данных (EDA). Ниже ссылки, описывающие взгляд на процесс EDA "сверху":

1. <https://waksoft.susu.ru/2021/07/23/issledovatel'skij-analiz-dannyh-v-python-rukovodstvo-dlya-novichkov-na-2021-god/>
(<https://waksoft.susu.ru/2021/07/23/issledovatel'skij-analiz-dannyh-v-python-rukovodstvo-dlya-novichkov-na-2021-god/>)
2. <https://www.kaggle.com/code/ekami66/detailed-exploratory-data-analysis-with-python> (<https://www.kaggle.com/code/ekami66/detailed-exploratory-data-analysis-with-python>)

3. <https://www.datacareer.de/blog/exploratory-data-analysis-in-python/>
(<https://www.datacareer.de/blog/exploratory-data-analysis-in-python/>).

Ввод [8]: `# проверка
for name, df in tables:
 display(df.head())`

	id	покупательская_активность	тип_сервиса	разрешить_сообщать	маркет_актив_6
0	215348	снизилась	премиум	да	
1	215349	снизилась	премиум	да	
2	215350	снизилась	стандартт	нет	
3	215351	снизилась	стандартт	да	
4	215352	снизилась	стандартт	нет	

	id	период	выручка
0	215348	препредыдущий_месяц	0.0
1	215348	текущий_месяц	3293.1
2	215348	предыдущий_месяц	0.0
3	215349	препредыдущий_месяц	4472.0
4	215349	текущий_месяц	4971.6

	id	период	минут
0	215348	текущий_месяц	14
1	215348	предыдцщий_месяц	13
2	215349	текущий_месяц	10
3	215349	предыдцщий_месяц	12
4	215350	текущий_месяц	13

	id	прибыль
0	215348	0,98
1	215349	4,16
2	215350	3,13
3	215351	4,87
4	215352	4,21

Проведем проверку значений в столбцах.

```
Ввод [9]: for name, df in tables:
            print(f"\n\033[1mТаблица: {name}\033[0m")
            print("-" * 40)

            for col in df.columns:
                print(f"\n{col}:")
                print("Уникальных:", df[col].nunique())

                if df[col].nunique() < 20:
                    print("Значения:", sorted(df[col].dropna().unique()))
```

Таблица: market_file

```
id:
Уникальных: 1300

покупательская_активность:
Уникальных: 2
Значения: ['прежний уровень', 'снизилась']

тип_сервиса:
Уникальных: 3
Значения: ['премиум', 'стандарт', 'стандартт']

разрешить_сообщать:
Уникальных: 2
Значения: ['да', 'нет']
```

```
Ввод [10]: # редактируем ошибки
market_file['тип_сервиса'] = market_file['тип_сервиса'].replace('станд', 'стандарт')
market_file['популярная_категория'] = market_file['популярная_категория'].replace('косметика и аксесуары', 'косметика и аксессуары')
market_time['период'] = market_time['период'].replace('предыдщий_месяц', 'предыдущий_месяц')

# и сразу же проверим
print(market_file['тип_сервиса'].unique())
print(market_file['популярная_категория'].unique())
print(market_time['период'].unique())

['премиум' 'стандарт']
['товары для детей' 'домашний текстиль' 'косметика и аксессуары'
 'техника для красоты и здоровья' 'кухонная посуда'
 'мелкая бытовая техника и электроника']
['текущий_месяц' 'предыдущий_месяц']
```

ОТЛИЧНО! 👍

Отлично, молодец, обнаружены 3 "разночтения" (**стандартт** и **предыдщий_месяц** и **Косметика и аксесуары**) - по факту в одних и тех же категориях в признаках.

Названия столбцов привели к единому стилю. А так же данные в датафреймах привели к нижнему регистру и провели редактирование.

Далее проверим датафреймы по типу данных, а так же на пропуски и дубликаты(в том числе неявные).

Ввод [11]:

```
for name, df in tables:
    print(name)
    print(df.dtypes)
    print('-' * 50)
```

```
market_file
id                                int64
покупательская_активность        object
тип_сервиса                       object
разрешить_сообщать               object
маркет_актив_6_мес               float64
маркет_актив_тек_мес             int64
длительность                     int64
акционные_покупки               float64
популярная_категория            object
средний_просмотр_категорий_за_визит int64
неоплаченные_продукты_штук_квартал int64
ошибка_сервиса                  int64
страниц_за_визит                int64
dtype: object
```

```
-----
market_money
id                                int64
период                           object
dtype: object
```

Ввод [12]:

```
# в целом все хорошо, но столбец "прибыль" в датафрейме money, изменим
money['прибыль'] = money['прибыль'].str.replace(',', '.').astype(float)
```

ОТЛИЧНО! 👍

Отлично, привели типы ... явно числовые типы, но при этом имеющие тип "строка" приведены к числовому типу.

Кроме того, что теперь признаки в нашем датафрейме имеют корректные типы данных - это действие ещё и вот по какой причине полезно: оптимизация объёма больших датасетов может помочь значительно сократить объём памяти (занимаемый датасетом). Это особенно важно, если у нас ограничены вычислительные ресурсы или требуется работать с огромными объёмами данных.

Функция `pd.to_numeric()` с параметром `downcast='integer'` позволяет снизить тип числовых данных до наименьшего возможного размера без потери информации. Например, преобразование целочисленных значений из типа `int64` до `int32`, `int16` или `int8` может существенно сократить использование памяти, особенно если значения в датасете могут быть представлены более короткими типами данных.

Однако при использовании этой оптимизации следует быть осторожным. Преобразование типов данных может привести к потере точности или диапазона значений, особенно если числа в датасете имеют большой разброс или могут выходить за пределы нового типа данных. Поэтому перед использованием

функции `pd.to_numeric()` с параметром `downcast='integer'`, рекомендуется проверить, что сокращение размера типа не приведет к потере важной информации для конкретной задачи.

```
Ввод [13]: # выводим пропуски
for name, df in tables:
    print('Пропуски:', name)
    print(df.isna().sum().sort_values(ascending=False))
    print('Дубликаты:', df.duplicated().sum())
    print('-' * 50)
```

```
Пропуски: market_file
id                0
покупательская_активность  0
тип_сервиса       0
разрешить_сообщать  0
маркет_актив_б_мес  0
маркет_актив_тек_мес  0
длительность      0
акционные_покупки  0
популярная_категория  0
средний_просмотр_категорий_за_визит  0
неоплаченные_продукты_штук_квартал  0
ошибка_сервиса     0
страниц_за_визит   0
dtype: int64
Дубликаты: 0
```

```
-----
Пропуски: market_money
id                0
сервис            0
```

Комментарий 🙋

В наших данных пропусков НЕТ, НО ЕСЛИ БЫ БЫЛИ я стараюсь дать подробный коммент-видение на тему обработки пропусков данных (так как понимаю, что несмотря на кажущуюся простоту - не очень просто понять чем ("заглушкой"? / медианой? / средним? / по комбинации столбцов?) заполнять? а может удалять?

По результатам проверки 5+ тысяч проектов за 3 года у меня родилось небольшое эссе на тему "удаления vs. спасения" данных, как раз на основе тех ситуаций и вопросов, которые мне коллеги задавали, которые мы обсуждали, уточняли и исправляли ...

► [Подробнее ...](#)

Идеальные датафреймы. Пропусков и дубликатов нет.

Комментарий 🙋

Хочу отметить, что в общем случае, "в жизни", даже если `df.duplicated().sum() == 0` - это может совсем не означать, что в данных нет дубликатов.

В реальных рабочих датасетах скорее всего не будет полных дубликатов, но вот неявные наверняка будут встречаться, соответственно с ними надо будет уметь работать. Для поиска неявных дублей обычно используют параметр `subset` у функции `duplicated()`. Правда здесь, в нашем проекте, надо поразмыслить над тем какие комбинации столбцов могут говорить о реальных дублях в данных.

```
Ввод [14]: # сделаем столбцы в датафрейме market_file покуп. активность, тип сервиса
le = LabelEncoder()
market_file['покупательская_активность'] = le.fit_transform(market_file['покупательская_активность'])
market_file['тип_сервиса'] = market_file['тип_сервиса'].map({'премиум': 1, 'стандарт': 0}).astype(int)
market_file['разрешить_сообщать'] = market_file['разрешить_сообщать'].map({'да': 1, 'нет': 0}).astype(int)
```

Необходимо исправить ❌

```
Ввод [45]: # сделаем столбцы в датафрейме market_file покуп. активность, тип сервиса и разрешение - бинарными.
market_file['покупательская_активность'] = market_file['покупательская_активность'].map({'снизилась': 1,
'прежний уровень': 0}).astype(int)
market_file['тип_сервиса'] = market_file['тип_сервиса'].map({'премиум': 1, 'стандарт': 0}).astype(int)
market_file['разрешить_сообщать'] = market_file['разрешить_сообщать'].map({'да': 1, 'нет': 0}).astype(int)
```

Кодирование таргета нельзя делать "руками":

- при появлении новых значений категориальных признаков, не предусмотренных твоим решением, модель или упадёт, или будет генерировать ошибки;
- такое решение не сделаешь частью пайплайна, а все современные решения базируются на них.

Поэтому кодирование нужно делать только с применением обучаемых трансформеров. Для кодирования целевого признака нужно использовать `LabelEncoder` (сокращаю до LE): <https://scikit-learn.org/stable/modules/generated/sklearn.preprocessing.LabelEncoder.html> (<https://scikit-learn.org/stable/modules/generated/sklearn.preprocessing.LabelEncoder.html>).

p.s. если стоит вопрос в том, что тебе не нравится, что LE, например, присваивает классу "снизилась" метку 0, а "прежний уровень" - метку 1, то значения меток можно инвертировать с помощью небольшого лайфхака:

```
import numpy as np

# Пример преобразованных классов
target = np.array([0, 1, 1, 0, 1, 0])

# Инвертирование классов
```

LE использован.

КОММЕНТАРИЙ V2

Исправлено.

Покупательская способность: снизилась - 1, прежний уровень - 0.

Тип сервиса: премиум - 1, стандарт - 0.

Разрешить сообщать: да - 1, нет - 0.

Ввод [15]: `# проверка`
`market_file.head()`

Out [15]:

	id	покупательская_активность	тип_сервиса	разрешить_сообщать	маркет_актив_6
0	215348	1	1	1	
1	215349	1	1	1	
2	215350	1	0	0	
3	215351	1	0	1	
4	215352	1	0	0	

Дубликатов, пропусков нет. Типы данных в порядке, значения привели к единому стилю.

ОТЛИЧНО! 👍

В этом разделе всё ОК: необходимые после загрузки данных действия (1. требования PER8, 2. дубликаты, 3. анализ уникальных значений в признаках, 4. пропуски) выполнены. Этот вывод результат твоей работы описывает и подытоживает. Здесь всё супер, молодец!

3. Исследовательский анализ данных

Проведем статистический анализ.

```
Ввод [16]: for name, df in tables:
            print(name)

            df_to_analyze = df.drop(columns=['id'], errors='ignore')

            num_cols = df_to_analyze.select_dtypes(include='number')
            if not num_cols.empty:
                display(num_cols.describe())

            cat_cols = df_to_analyze.select_dtypes(exclude='number')
            if not cat_cols.empty:
                display(cat_cols.describe())

            print('-' * 50)
```

market_file

	покупательская_активность	тип_сервиса	разрешить_сообщать	маркет_актив_6_мес
count	1300.000000	1300.000000	1300.000000	1300.000000
mean	0.383077	0.289231	0.740000	4.253769
std	0.486324	0.453580	0.438803	1.014814
min	0.000000	0.000000	0.000000	0.900000
25%	0.000000	0.000000	0.000000	3.700000
50%	0.000000	0.000000	1.000000	4.200000
75%	1.000000	1.000000	1.000000	4.900000
max	1.000000	1.000000	1.000000	6.600000

популярная_категория	
count	1300

Данные неплохие, в таблице market_money наблюдаем выброс, редактируем путем удаления первых двух цифр.

ОТЛИЧНО! 👍

Молодец! Не часто вижу чтобы результат describe() вообще, в принципе, комментировали. Обычно почему-то просто "выводят эту табличку" и ни словом в выводах её не упоминают.

describe() играет важную роль в понимание распределения статистик признаков в данных. Отсюда можно увидеть аномалии, выбросы. Просто "посмотреть" на эти таблички недостаточно, обязательно нужен вывод по анализу результата describe().

describe() в pandas - это мощный метод, который предоставляет статистическое описание данных. Вот несколько причин, почему describe() важен:

1. Общее понимание данных: describe() предоставляет общий обзор статистических характеристик, таких как среднее значение, стандартное отклонение, минимум, максимум и квартили. Это позволяет быстро понять основные характеристики данных.

2. Выявление выбросов: Анализ статистических показателей может помочь выявить аномалии или выбросы в данных. Например, если среднее значение сильно отличается от медианы, это может указывать на наличие выбросов.
3. Понимание распределения данных: `describe()` также предоставляет информацию о распределении данных через квантили, что может быть полезным для понимания формы распределения.
4. Быстрый анализ пропущенных значений: Метод также выводит количество непустых значений для каждого столбца, что может помочь в определении пропущенных данных.

Ввод [17]: `market_money['выручка'] = market_money['выручка'].replace(106862.200000
и сразу же проверим
display(market_money['выручка'].describe()))`

```
count    3900.000000
mean     5000.055026
std       707.538134
min        0.000000
25%      4590.150000
50%      4957.500000
75%      5363.000000
max       7799.400000
Name: выручка, dtype: float64
```

ОТЛИЧНО! 👍

Отлично, выброс обработан!

Ввод [18]: `print(market_money[market_money['id'] == 215380])`

```
      id      период  выручка
96  215380  препредыдущий_месяц  5051.0
97  215380    предыдущий_месяц  6077.0
98  215380      текущий_месяц  6862.2
```

Ввод [19]: `market_file.head()`

Out [19]:

	id	покупательская_активность	тип_сервиса	разрешить_сообщать	маркет_актив_6
0	215348	1	1	1	
1	215349	1	1	1	
2	215350	1	0	0	
3	215351	1	0	1	
4	215352	1	0	0	

Далее построим диаграммы распределения.

```

Ввод [20]: cat_cols = ['популярная_категория']

discrete_cols = ['покупательская_активность', 'тип_сервиса', 'разрешит_
                'средний_просмотр_категорий_за_визит', 'неоплаченные_
                'ошибка_сервиса', 'страниц_за_визит']

continuous_cols = ['маркет_актив_6_мес', 'акционные_покупки', 'длительн

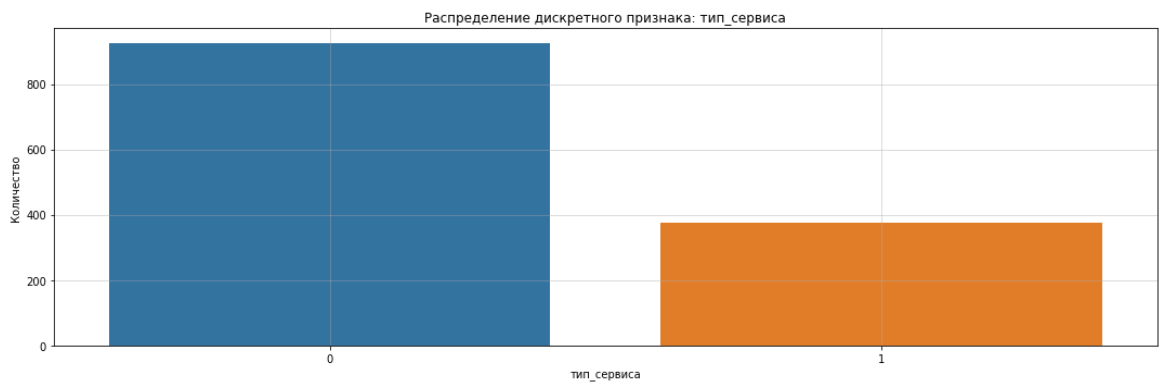
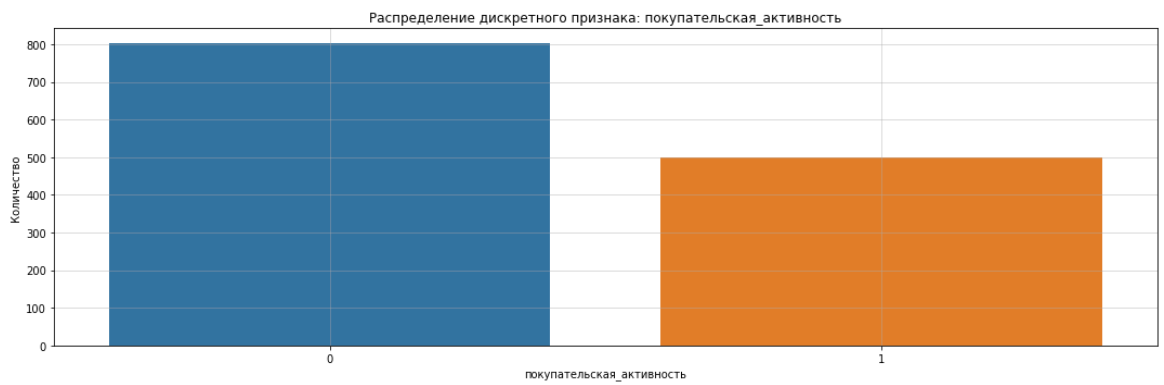
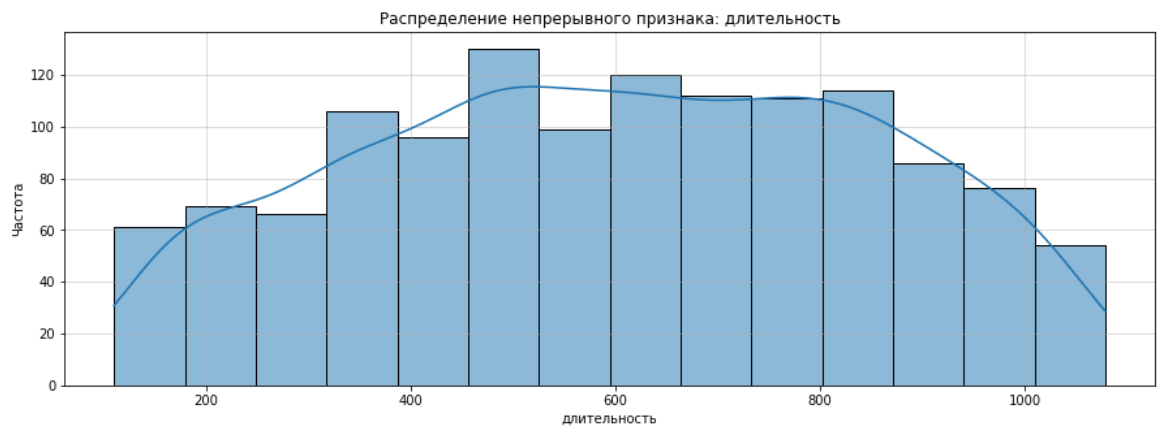
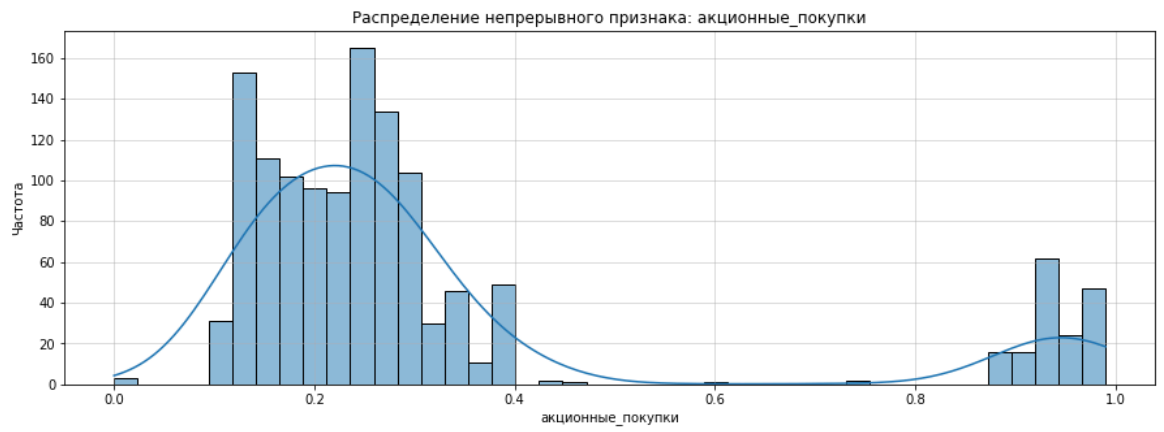
for col in continuous_cols:
    plt.figure(figsize=(15, 5))
    sns.histplot(
        data=market_file[col],
        kde=True,
        bins='auto'
    )
    plt.grid(True, linestyle='-', alpha=0.5)
    plt.title(f"Распределение непрерывного признака: {col}")
    plt.xlabel(col)
    plt.ylabel('Частота')
    plt.show()

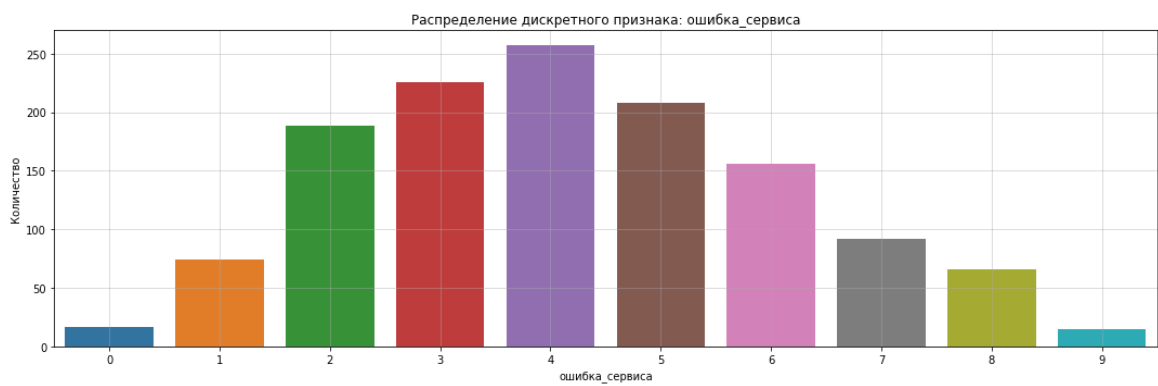
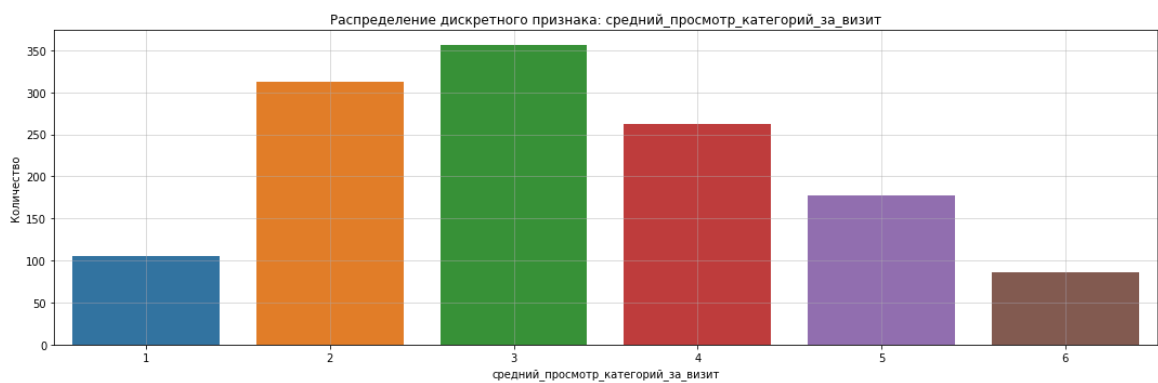
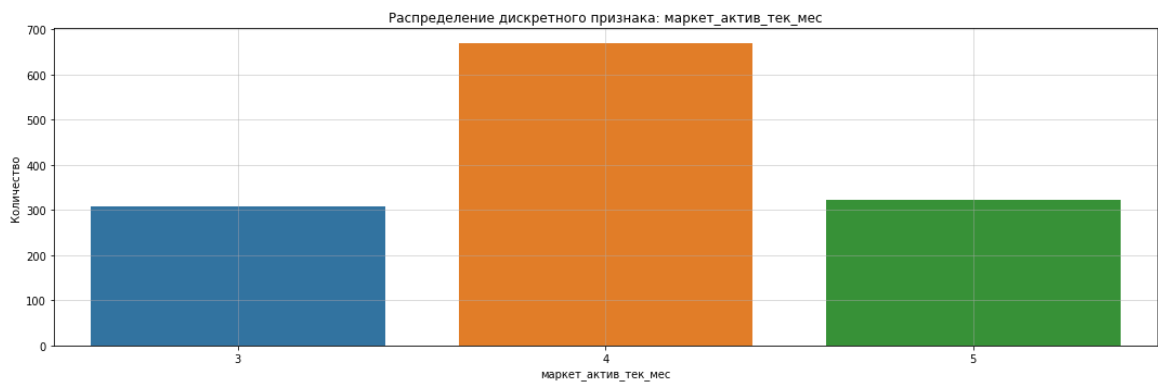
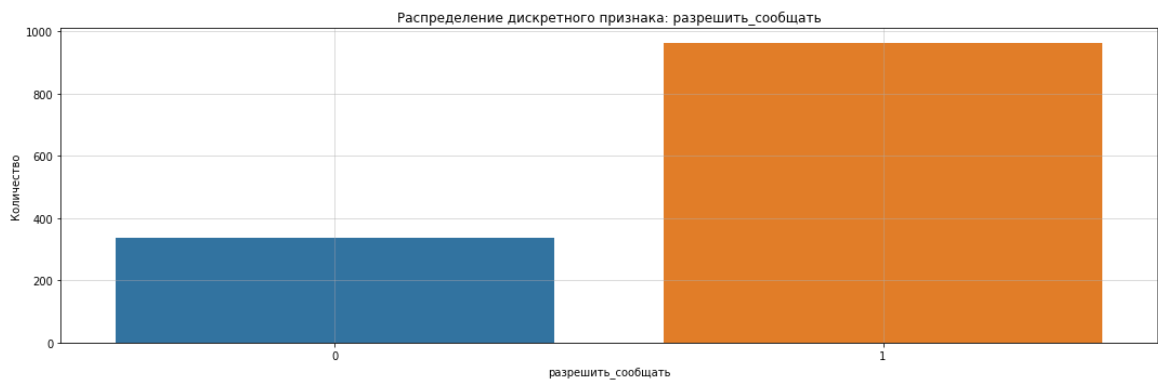
for col in discrete_cols:
    plt.figure(figsize=(15, 5))
    sns.countplot(
        x=market_file[col]
    )
    plt.grid(True, linestyle='-', alpha=0.5)
    plt.title(f"Распределение дискретного признака: {col}")
    plt.xlabel(col)
    plt.ylabel('Количество')
    plt.tight_layout()
    plt.show()

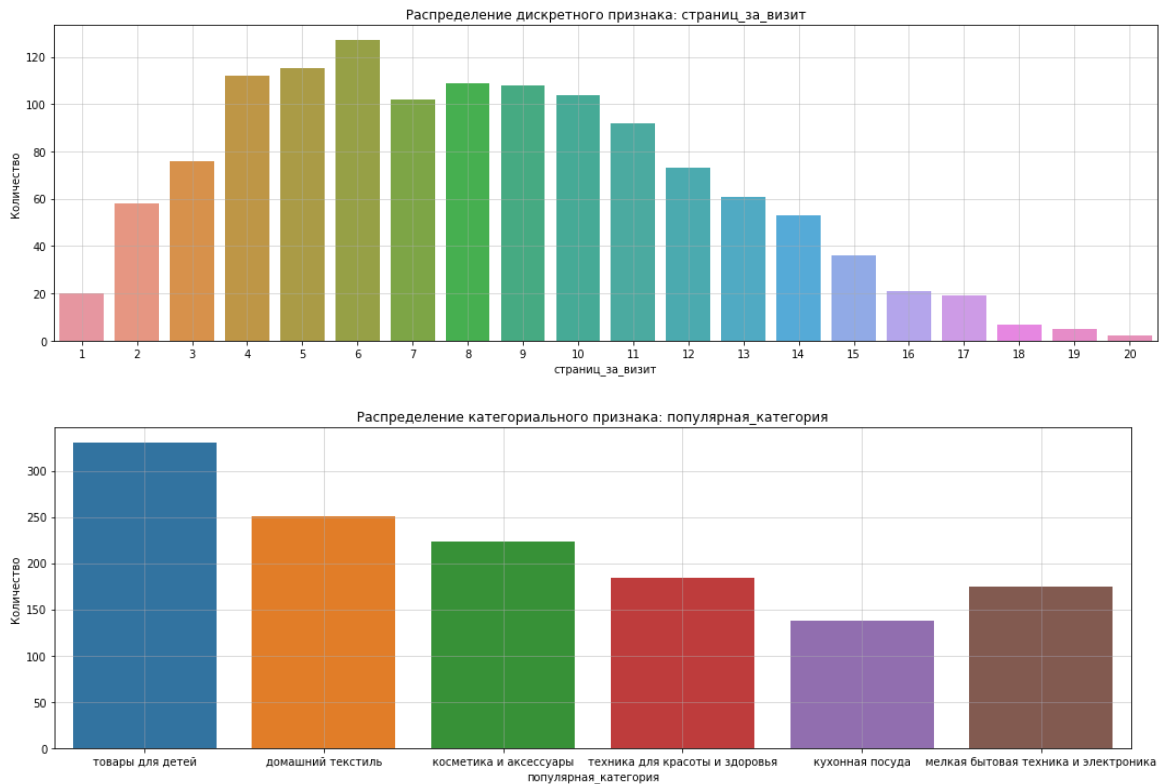
for col in cat_cols:
    plt.figure(figsize=(15, 5))
    sns.countplot(
        x=market_file[col]
    )
    plt.grid(True, linestyle='-', alpha=0.5)
    plt.title(f"Распределение категориального признака: {col}")
    plt.xlabel(col)
    plt.ylabel('Количество')
    plt.tight_layout()
    plt.show()

```









ОТЛИЧНО! 👍

Просто супер: графики подписаны, оси подписаны. Не часто на самом деле такой скрупулёзный подход к оформлению встречается. Молодец!

Я закончил физико-математический факультет Петрозаводского государственного университета. Нас там с первого курса приучили - если на сдачу лабораторной работы по физике приходишь с графиком, на котором не подписано общее название, названия осей (с единицами измерений), единичная шкала - всё сдача заканчивается не начавшись. Или если у моей лабораторной работы отсутствует вывод: что делали? по какой методике? что "намеряли"? как интерпретировали? какой финальный вывод? - всё, тоже на выход. Ну и в принципе сейчас, на текущей работе: код, который написан в тетрадке мало кому понятен и не сильно кому интересен. Важнее детальные выводы и тренды, которые попадут на слайд презентации в PowerPoint. Именно поэтому столь важное значение в аналитике данных придается промежуточным выводам и уж тем более финальному.

(МЕТОДИЧЕСКИЕ МАТЕРИАЛЫ ПО ОФОРМЛЕНИЮ ГРАФИКОВ):

При работе с графиками важно соблюдать следующие правила:

1. Стоит подписать названия осей X и Y.
2. Также стоит дать название графику.
3. Технические комментарии типа `<AxesSubplot: xlabel='datetime'>` стоит убирать - это можно сделать, поставив точку с запятой в последней строке кода в блоке визуализации, либо сделать последней строкой кода этого блока метод `plt.show()`.
4. Графики нужно сделать больше - графики дефолтного размера не очень удобны для чтения. Удобнее будет настроить параметр `plt.rcParams["figure.figsize"] = (x,y)` сразу в ячейке импорта библиотек, и все

графики проекта будут одного размера.

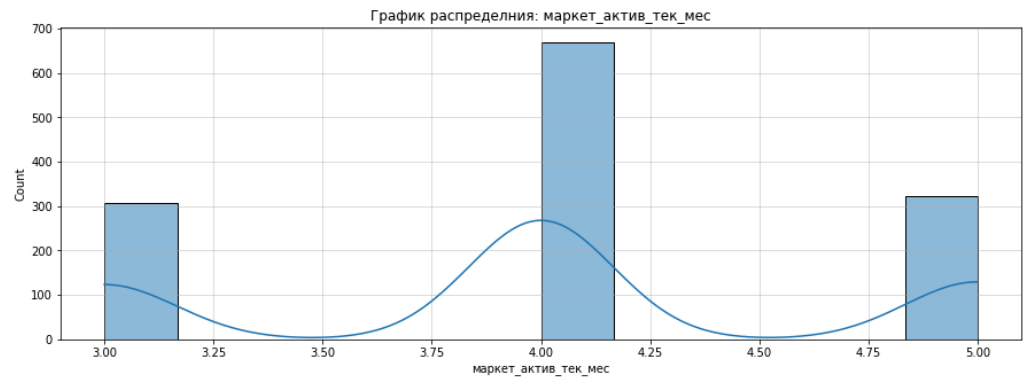
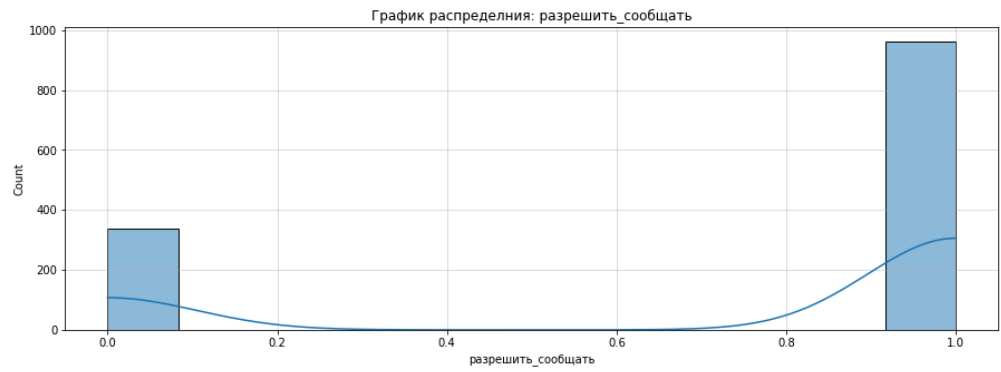
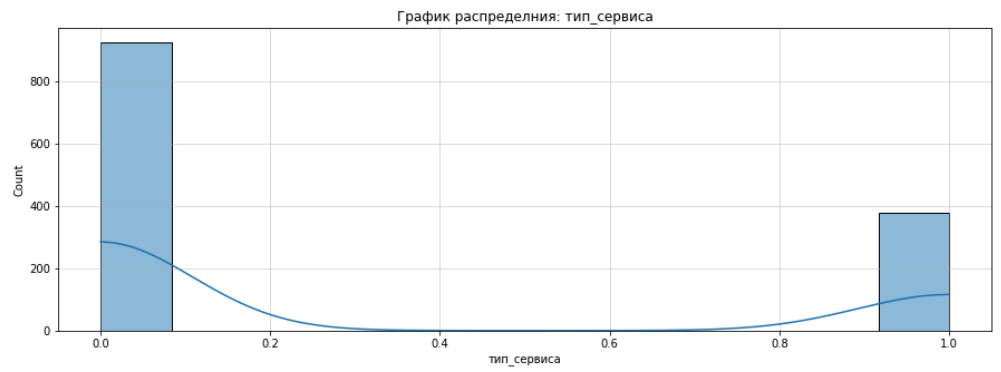
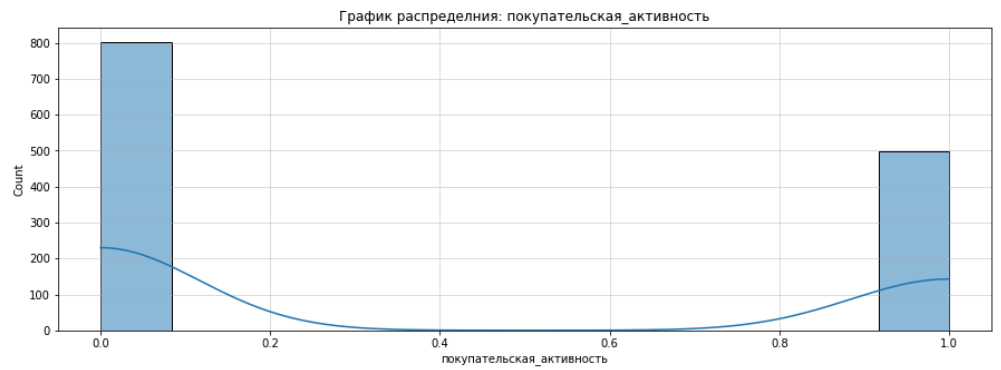
Несколько таких нехитрых приёмов повысят качество твоих визуализаций в проекте. **И здесь следует сказать что ты об правилах оформления всё знаешь! МОЛОДЕЦ!**

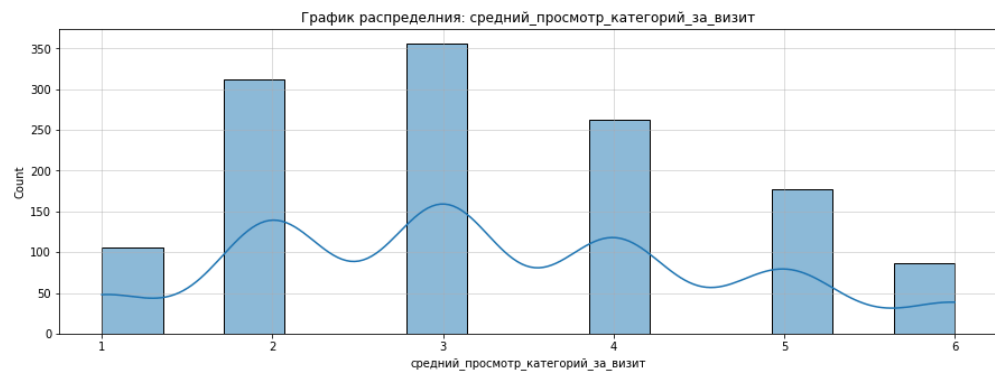
Комментарий 👉

Хотя на графиках подписи есть (об этом похвальный комментарий выше), но я всё-таки хочу обратить твое внимание, что одинаковые "обезличенные" подписи на всех графиках - это не ОК. Графики принято подписывать так, чтобы подписи **отражали суть** изображенного на осях, а также подписи должны содержать и единицы измерения того, что на осях изображено.

Если графики строятся в цикле, то выход такой: вручную создать словарь: "ключ" - название признака по которому на соответствующем шаге будет строиться график, "значения" - список названий ([название графика, название оси ОХ, название оси ОУ]). Далее пробегаем в цикле по названиям признаков (в данных). Сопоставляем название признака с ключом словаря, а названия подписей (графика и 2х осей) берем из соответствующего этому ключу значения.

Необходимо исправить ❌

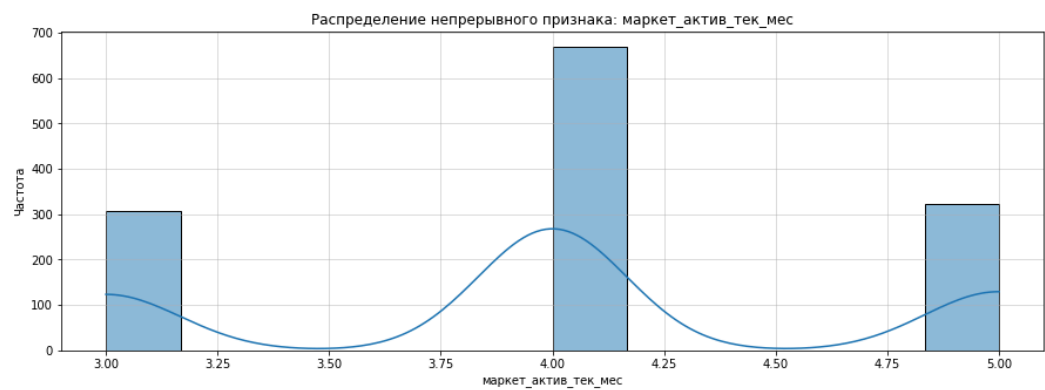


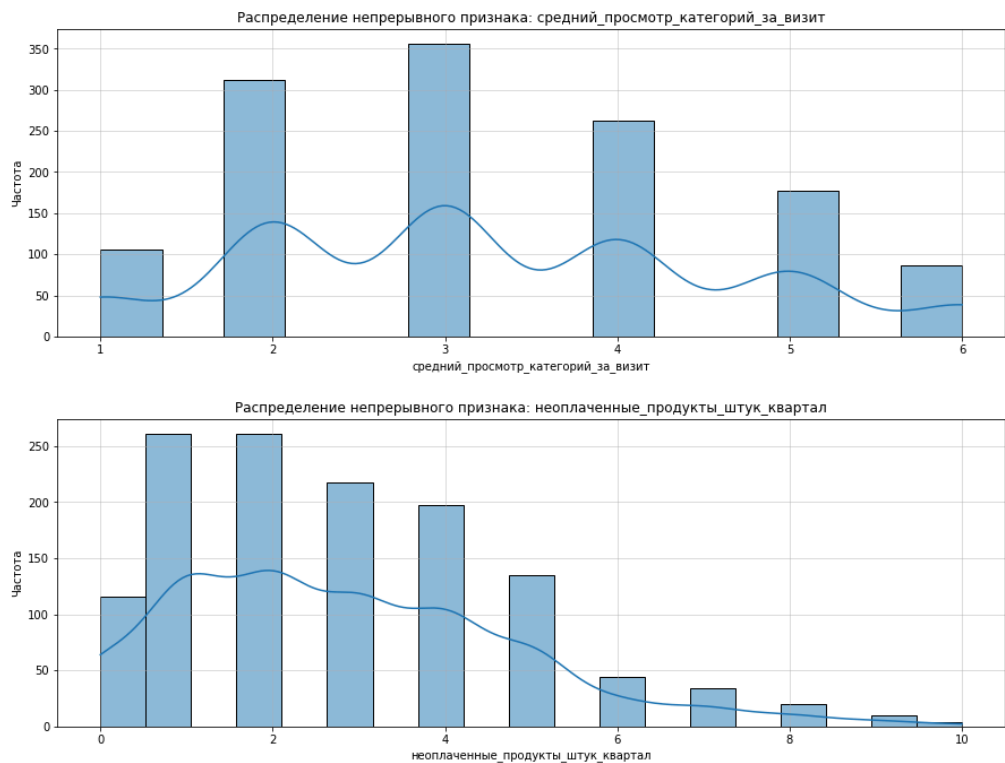


Отредактировал.

КОММЕНТАРИЙ V2

Артемий - здесь пока полного исправления не вижу. Вот эти признаки - они дискретные, но их визуализация выполняется гистограммами:





Отредактировал.

КОММЕНТАРИЙ V3

Исправлено.

Выводы по market_file:

- покупательская активность снизилась значительно.
- тип сервиса преобладает "стандарт".
- большинство пользователей дают согласие на рекламу.
- за последние 6 месяцев, в среднем было 4 маркентиговых активности на 1 пользователя.
- в текущем месяце есть значения 3, 4 и 5 активностей, преобладают так же 4. Маркетинговая стратегия не изменилась за последние пол года.
- "жизненный цикл" клиента или длительность с момента регистрации - распределение почти равномерное.
- большинство клиентов берут со скидкой от 10% до 40% и при этом есть небольшая группа, с покупками со скидкой районе 99%.
- в среднем количество просматриваемых категорий от 2 до 4.
- от 1 до 5, в среднем колличестве неоплаченных позиций в корзине.
- число сбоев наибольшее 4.
- в среднем пользователь смотрит до 15 страниц.
- самая популярная категория: "товары для детей", наименее популярная: "кухонная посуда".

ОТЛИЧНО! 👍

Данные загружены, отлично.

Хорошо, что применяешь метод `.head().sample()` таким образом в память не будет загружен весь датасет, объем которого может быть огромным.

Методы `.info()`, `.describe()`, `.value_counts()`, `.unique()` как правило часто встречаются и обязательны к применению в задачах по исследованию данных, ведь с их помощью можно получить первые инсайты.

Итак, мы провели предварительный анализ данных, проверили данные на наличие пропусков и затем обработали их (аргументированно заполнили / удалили / оставили как есть). Провели графический анализ. Сделали и озвучили выводы. Теперь можно приступить собственно к моделированию и ответам на вопросы проекта.

ОТЛИЧНО! 👍

Этот шаг спасет твою аналитику (и карьеру): классический кейс - к аналитику (неважно: DA-аналитику, DS-аналитику, ML-инженеру) прибегает его начальник с горящими глазами: «Срочно нужно посчитать метрики, отчет нужен вчера, Я ЖДУУУУУУУ!!!!!!!!!!!!!!!!!!!!!!». Аналитик лезет в сырые данные, быстро собирает дашборд ... и всё. Ни тебе проверки распределений, ни поиска выбросов, ни визуализации. Начальник на следующий день презентует эти цифры как истину в последней инстанции. А потом на них строятся бизнес-решения стоимостью десятки-сотни миллионов рублей ...

► **ТАК ДЕЛАТЬ НЕЛЬЗЯ! НИКОГДА!!!**

```

Ввод [21]: # market_money
market_money_group_period = market_money.groupby('период')['выручка'].
market_money_group_period = market_money_group_period.reset_index()
market_money_group_period.columns = ['период', 'выручка']
period_order = ['препредыдущий_месяц', 'предыдущий_месяц', 'текущий_м
market_money_group_period['период'] = pd.Categorical(
    market_money_group_period['период'],
    categories=period_order,
    ordered=True)

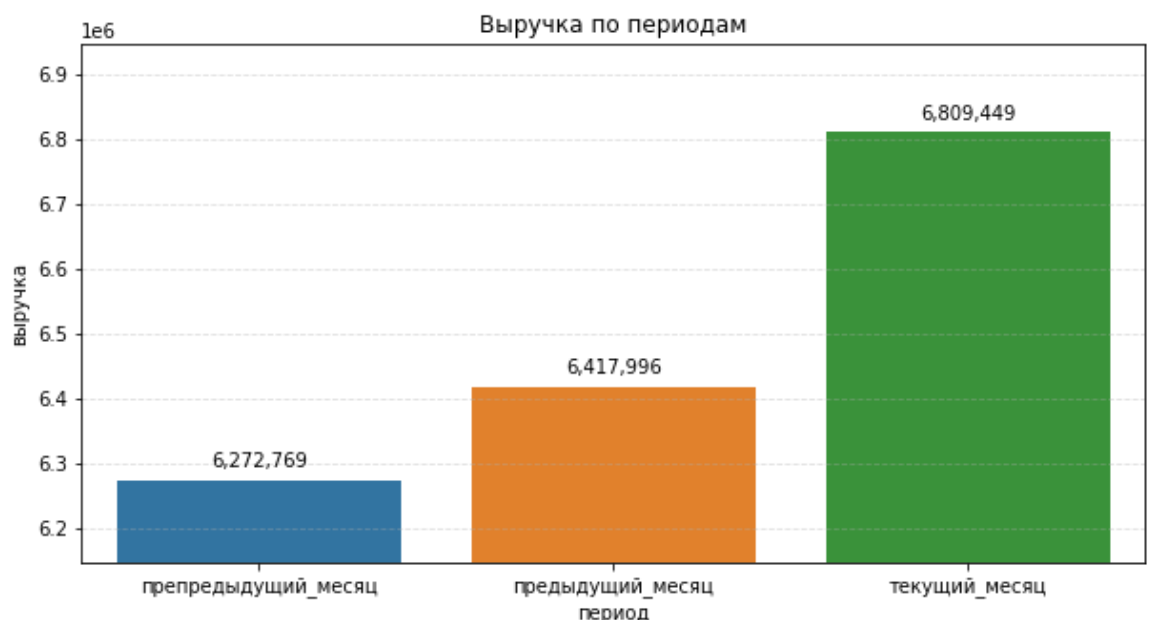
plt.figure(figsize=(10, 5))
ax = sns.barplot(
    data=market_money_group_period,
    x='период',
    y='выручка')

y_min = market_money_group_period['выручка'].min() * 0.98
y_max = market_money_group_period['выручка'].max() * 1.02
plt.ylim(y_min, y_max)

for p in ax.patches:
    ax.annotate(
        f"{p.get_height():,.0f}",
        (p.get_x() + p.get_width() / 2., p.get_height()),
        ha='center',
        va='center',
        xytext=(0, 10),
        textcoords='offset points')

plt.title('Выручка по периодам')
plt.grid(axis='y', linestyle='--', alpha=0.4)
plt.show()

```



Ввод [22]: `# проверим количество активных покупателей в каждом периоде`
`active_buyers = market_money[market_money['выручка'] > 0].groupby('пер`
`print(active_buyers)`

```
период
предыдущий_месяц    1297
препредыдущий_месяц 1297
текущий_месяц        1300
Name: id, dtype: int64
```

ОТЛИЧНО! 👍

Супер! Здесь молодец, НЕ УПУСКАЕШЬ требование брифа в части обработки покупателей НЕ имеющих полной покупательской истории в крайние 3 месяца:

Шаг 3. Исследовательский анализ данных

Проведите исследовательский анализ данных из каждой таблицы. Отберите клиентов с покупательской активностью не менее трёх месяцев, то есть таких, которые что-либо покупали в этот период. Оформите выводы по результатам шага.

Выручка растет каждый месяц, при том же количестве покупателей.

4. Объединение таблиц

Развернем таблицы по периодам. Далее объединим по столбцы 'id'.

Ввод [23]: `money_pivot = market_money.pivot(`
 `index='id',`
 `columns='период',`
 `values='выручка'`
`).add_prefix('выручка_').reset_index()`

`money_pivot.columns = ['id', 'выручка_препред', 'выручка_пред', 'выруч`

Ввод [24]: `time_pivot = market_time.pivot(`
 `index='id',`
 `columns='период',`
 `values='минут'`
`).add_prefix('время_').reset_index()`

`time_pivot = time_pivot.rename(columns={'время_предыдущий_месяц': 'вре`

```
Ввод [25]: final_df = (
    market_file
    .merge(money_pivot, on='id', how='left')
    .merge(time_pivot, on='id', how='left')
)
# Удаляем пользователей без полной истории покупок
active_users = market_money.groupby('id')['период'].nunique()
active_users = active_users[active_users == 3].index.tolist()

final_df = final_df[final_df['id'].isin(active_users)]
display(final_df.head())
```

	id	покупательская_активность	тип_сервиса	разрешить_сообщать	маркет_актив_6
0	215348	1	1	1	
1	215349	1	1	1	
2	215350	1	0	0	
3	215351	1	0	1	
4	215352	1	0	0	

ОТЛИЧНО! 👍

Есть, отлично! Из "длинного" датафрейма сделали "широкий". Ровно то, что и требуется:

Шаг 4. Объединение таблиц

4.1 Объедините таблицы `market_file.csv`, `market_money.csv`, `market_time.csv`. Данные о прибыли из файла `money.csv` при моделировании вам не понадобятся.

4.2 Учитывайте, что данные о выручке и времени на сайте находятся в одном столбце для всех периодов. В итоговой таблице сделайте отдельный столбец для каждого периода.

Необходимо исправить ❌

Из финальной версии датафрейма "неактивные пользователи" удалены? Если нет, то это следует сделать, Артемий.

Отредактировал.

КОММЕНТАРИЙ V2

Исправлено.

Развернули таблицы по периода, для удобства и объединили.

Комментарий 🙌

В случае объединения/слияния датафреймов рекомендую тебе смотреть на размерности датафреймов ДО слияния и ПОСЛЕ него. В подобных операциях часто ошибки бывают.

5. Корреляционный анализ

ОТЛИЧНО! 👍

Смотришь скоррелированность признаков - отлично! Об этом чуть ниже подробнее напишу....

p.s. присмотришься к библиотеке **phik**! Она (Phi_K correlation) представляет собой инструмент для вычисления и визуализации корреляции между категориальными и количественными переменными. Она может быть полезной в анализе данных перед построением моделей машинного обучения, так как позволяет обнаружить скрытые взаимосвязи между переменными, которые могли бы остаться незамеченными при обычных методах корреляционного анализа. Преимущества использования **phik**:

1. **Учет категориальных переменных:** Традиционные методы вычисления корреляции, такие как коэффициент корреляции Пирсона, применяются только к количественным переменным. **phik** позволяет вычислять корреляцию между категориальными и количественными переменными, что делает его мощным инструментом для анализа разнообразных типов данных.
2. **Обнаружение нелинейных зависимостей:** **phik** способен обнаруживать нелинейные связи между переменными, которые могли бы быть упущены традиционными методами. Это особенно важно в анализе данных, где зависимости могут быть сложными и нелинейными (что может влиять на твой выбор признаков для моделей МО).
3. **Подходит для больших наборов данных:** **phik** обладает хорошей масштабируемостью, что позволяет использовать его для анализа больших наборов данных без серьезного снижения производительности.
4. **Визуализация результатов:** Библиотека предоставляет инструменты для визуализации матрицы корреляции между переменными, что помогает лучше понимать зависимости в данных.

Вот пример кода, как использовать **phik** для вычисления корреляции:

```
# Установим "руками", если не установлена
!pip install phik -q

# Импортируем
import phik
import pandas as pd

# Загрузка данных
data = pd.read_csv('data.csv')

# Вычисление матрицы корреляции с использованием phik
corr_matrix = data.phik_matrix(interval_cols= ...)

# Визуализация матрицы корреляции
import seaborn as sns
import matplotlib.pyplot as plt

plt.figure(figsize=(10, 8))
sns.heatmap(corr_matrix, annot=True, cmap='coolwarm')
plt.title('Phi_K Correlation Matrix')
plt.show()
```

p.s. при использовании **phik** крайне важно настроить параметр **interval_cols**, передав ему список с названиями **непрерывных признаков**: если этого не сделать, библиотека своими силами будет пытаться разнести признаки по типам, и она зачастую делает в этом ошибки - от этого в свою очередь мы получаем некорректную оценку корреляций, так как от типа признака зависит метод оценки корреляции.

p.p.s. у этой библиотеки есть ограничение с которым я например сталкивался:

Проведем корреляционный анализ.

```

Ввод [26]: columns_to_drop = ['id', 'выручка_препред', 'выручка_пред', 'время_пре
final_df_no_id = final_df.drop(columns=columns_to_drop)

continuous_for_phik = [
    'маркет_актив_6_мес', 'длительность', 'акционные_покупки',
    'выручка_тек', 'время_текущий_месяц', 'прибыль'
]

available_continuous = [f for f in continuous_for_phik if f in final_c
print("Непрерывные признаки для phik:", available_continuous)

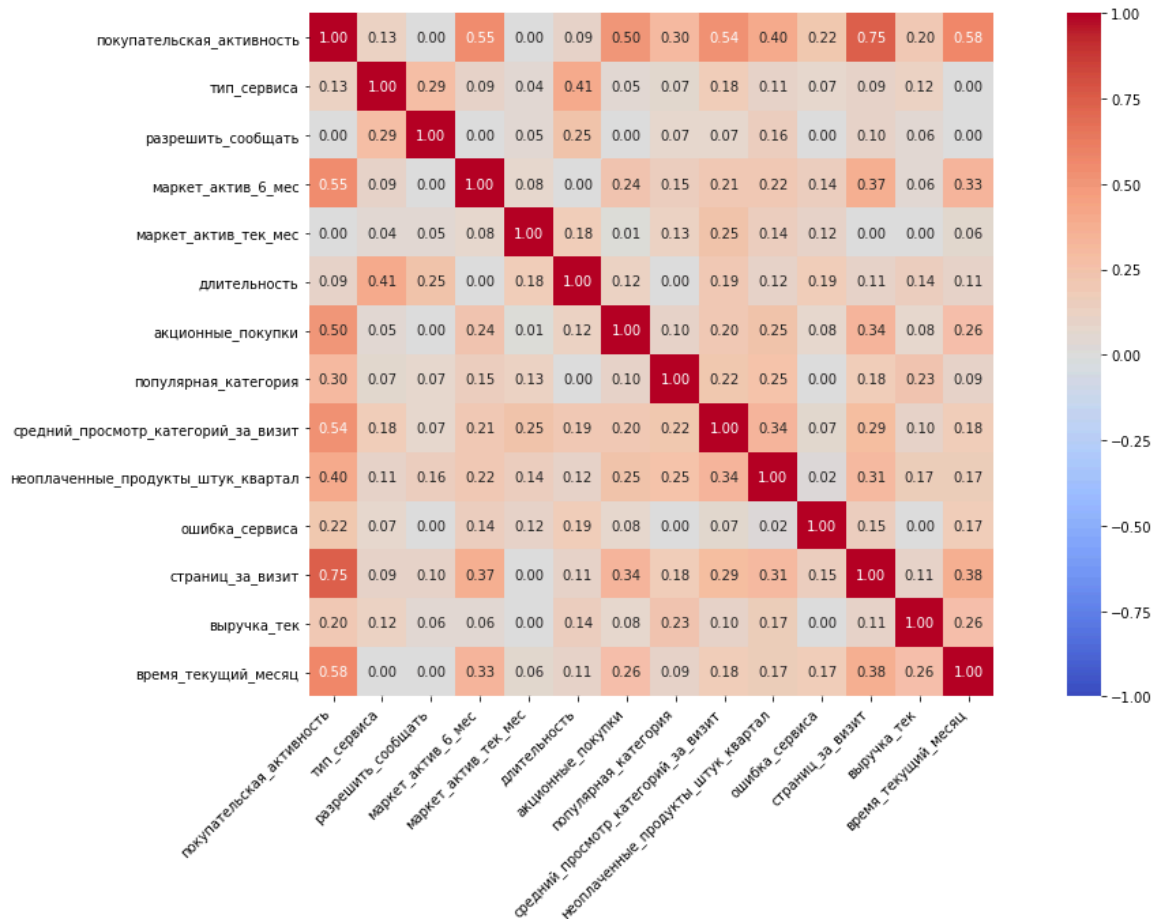
corr_matrix = final_df_no_id.phik_matrix(interval_cols=available_conti

plt.figure(figsize=(16, 10))
sns.heatmap(
    data=corr_matrix,
    annot=True,
    fmt='0.2f',
    vmin=-1,
    vmax=1,
    cmap='coolwarm',
    center=0,
    square=True
)
plt.title('Матрица корреляции Phi-K (Phik) между признаками', fontsize
plt.xticks(rotation=45, ha='right')
plt.yticks(rotation=0)
plt.tight_layout()
plt.show()

```

Непрерывные признаки для phik: ['маркет_актив_6_мес', 'длительность', 'акционные_покупки', 'выручка_тек', 'время_текущий_месяц']

Матрица корреляции Phi-K (Phik) между признаками



Необходимо исправить ❌

```
Ввод [56]: final_df_no_id = final_df.drop(columns = ['id'])
corr_matrix = final_df_no_id.corr()
```

1. Матрица корреляций Пирсона не подходит для анализа датасетов, в которых присутствуют категориальные или дискретные признаки - по ним корреляций Пирсона не может быть оценена вообще (в случае категориальных) или не может быть оценена корректно (в случае дискретных).
2. Нормально ли распределены наши признаки? Если нет, то каким методом надо считать коэффициенты корреляции? Небольшая напоминка на скрине: Пирсон - это дефолтный метод.

pandas.DataFrame.corr

DataFrame.corr(**method='pearson'**, min_periods=1, numeric_only=False) [\[source\]](#)

Compute pairwise correlation of columns, excluding NA/null values.

Parameters:

method : {'pearson', 'kendall', 'spearman'} or callable

Method of correlation:

- pearson : standard correlation coefficient

Отредактировал.

КОММЕНТАРИЙ V2

Необходимо исправить ❌

```
Ввод [26]: final_df_no_id = final_df.drop(columns = ['id'])  
num_columns = final_df_no_id.select_dtypes(include=['number']).columns.tolist()  
corr_matrix = final_df_no_id.phik_matrix(interval_cols=num_columns)
```

Все ли признаки в interval_cols сейчас действительно непрерывные? Или "затесались" в список ещё и дискретные?

Отредактировал.

КОММЕНТАРИЙ V3

Исправлено.

Видим, что у **покупательской активности** корреляция с:

- маркет_активбмесяц
- акционные_покупки
- средний просмотр_категорий_за_визит
- неоплаченные_продукты_штук_квартал
- страниц_за_визит
- выручка_пред
- время_пред
- время_текущий_месяц

Сильной мультиколлинеарности - нет.

Комментарий 🙌

Несколько моментов "на будущее" (сейчас забегаю чуть вперед, но будущем это будет важно) которые я хотел озвучить перед переходом к собственно исследованию (некоторые из этих моментов в твоём проекте реализованы):

- имеет смысл критично посмотреть на данные с точки зрения наличия пропусков в признаках (вопрос: все ли ml-модели "любят" пропуски?) и обработать или удалить их
- имеет смысл явно обозначить какую задачу мы будем решать: с учителем / без учителя, классификации / регрессии ...
- в моделях бинарной классификации (это ровным счётом наш случай!) имеет смысл смотреть на баланс классов. Два взгляда (с разных сторон) на эту проблему тут: <https://alexanderdyakonov.wordpress.com/2021/05/27/imbalance/>

(<https://alexanderdyakonov.wordpress.com/2021/05/27/imbalance/>) и <https://habr.com/ru/post/349078/> (<https://habr.com/ru/post/349078/>).

- имеет смысл смотреть на корреляцию признаков. При высокой их степени коррелированности возникает проблема мультиколлинеарность признаков. Проблема "мультиколлинеарности" это и про переобучение модели и про её интерпретируемость и про сложность модели (содержание в модели избыточного количества переменных). В чем негативный эффект сложной модели? Вот в чем: усложняется интерпретация параметров, оценки параметров ненадежны – получаются большие стандартные ошибки, которые меняются с изменением объема наблюдений, что делает модель регрессии непригодной для прогнозирования.

► [Подробнее ...](#)

Ввод [27]: `final_df.head()`

Out [27]:

	id	покупательская_активность	тип_сервиса	разрешить_сообщать	маркет_актив_6_
0	215348	1	1	1	
1	215349	1	1	1	
2	215350	1	0	0	
3	215351	1	0	1	
4	215352	1	0	0	

6. Использование пайплайнов

Ввод [28]:

```
# Обоснование выбора метрики ROC-AUC
print("""
ОБОСНОВАНИЕ ВЫБОРА МЕТРИКИ ROC-AUC:
1. Дисбаланс классов: в наших данных распределение целевой переменной
   неравномерно
2. ROC-AUC показывает качество разделения классов независимо от порога
   классификации
3. Метрика устойчива к дисбалансу классов и хорошо подходит для бинарной
   классификации
4. Позволяет оценивать модель на различных порогах принятия решений
""")
```

ОБОСНОВАНИЕ ВЫБОРА МЕТРИКИ ROC-AUC:

1. Дисбаланс классов: в наших данных распределение целевой переменной неравномерно
2. ROC-AUC показывает качество разделения классов независимо от порога классификации
3. Метрика устойчива к дисбалансу классов и хорошо подходит для бинарной классификации
4. Позволяет оценивать модель на различных порогах принятия решений

Ввод [29]:

```
# Импортируем необходимые библиотеки
import warnings
from scipy import stats
import numpy as np

# Подавляем конкретное предупреждение
warnings.filterwarnings("ignore", category=FutureWarning, module="sklearn")

# ШАГ 6: ПОДГОТОВКА ДАННЫХ И МОДЕЛИРОВАНИЕ

# Фиксируем random_state для воспроизводимости результатов
RANDOM_STATE = 42
TEST_SIZE = 0.25

# Признаки, которые исключаем из анализа
excluded_features = ['id', 'страниц_за_визит', 'время_пред', 'маркет_сегмент']

# Определяем все возможные признаки
categorical_features = ['тип_сервиса', 'разрешить_сообщать', 'популярная_категория']
numerical_features = [
    'маркет_актив_6_мес', 'длительность', 'акционные_покупки',
    'средний_просмотр_категорий_за_визит', 'неоплаченные_продукты_штук_квартал',
    'ошибка_сервиса', 'выручка_тек', 'время_текущий_месяц'
]

# Проверяем, какие признаки действительно есть в DataFrame
existing_columns = final_df.columns.tolist()
available_categorical = [f for f in categorical_features if f in existing_columns]
available_numerical = [f for f in numerical_features if f in existing_columns]

print("Доступные категориальные признаки:", available_categorical)
print("Доступные числовые признаки:", available_numerical)

# РАЗДЕЛЯЕМ ПРИЗНАКИ НА ГРУППЫ ДЛЯ РАЗНЫХ ТРАНСФОРМЕРОВ:
# 1. Числовые признаки для StandardScaler (стабильные без выбросов)
numerical_std = [
    'маркет_актив_6_мес', 'длительность', 'акционные_покупки',
    'средний_просмотр_категорий_за_визит', 'ошибка_сервиса', 'время_текущий_месяц'
]

# 2. Числовые признаки для RobustScaler (с потенциальными выбросами)
numerical_robust = [
    'неоплаченные_продукты_штук_квартал', 'выручка_тек'
]

# 3. Категориальные признаки для OrdinalEncoder (ранговый признак)
categorical_ordinal = ['тип_сервиса', 'разрешить_сообщать']

# 4. Категориальные признаки для OneHotEncoder (номинальные признаки)
categorical_ohe = ['популярная_категория'] # Номинальные признаки

# Проверяем, что все признаки доступны
numerical_std = [f for f in numerical_std if f in available_numerical]
numerical_robust = [f for f in numerical_robust if f in available_numerical]
categorical_ordinal = [f for f in categorical_ordinal if f in available_categorical]
categorical_ohe = [f for f in categorical_ohe if f in available_categorical]

print("\nГруппы для трансформеров:")
print("Numerical (StandardScaler):", numerical_std)
print("Numerical (RobustScaler):", numerical_robust)
print("Categorical (OrdinalEncoder):", categorical_ordinal)
```

```

print("Categorical (OneHotEncoder):", categorical_oh)

ordinal_categories = {
    'тип_сервиса': [0, 1],
    'разрешить_сообщать': [0, 1]
}

# СОЗДАЕМ СЛОЖНЫЙ COLUMNTRANSFORMER С РАЗНЫМИ ТРАНСФОРМЕРАМИ
preprocessor = ColumnTransformer(
    transformers=[
        # Пайплайн для стабильных числовых признаков (StandardScaler)
        ('num_std', Pipeline(steps=[
            ('imputer', SimpleImputer(strategy='median')),
            ('scaler', StandardScaler())
        ]), numerical_std),

        # Пайплайн для числовых признаков с выбросами (RobustScaler)
        ('num_robust', Pipeline(steps=[
            ('imputer', SimpleImputer(strategy='median')),
            ('scaler', RobustScaler())
        ]), numerical_robust),

        # Пайплайн для ранговых категориальных признаков (OrdinalEncoder)
        ('cat_ordinal', Pipeline(steps=[
            ('imputer', SimpleImputer(strategy='most_frequent')),
            ('encoder', OrdinalEncoder(
                categories=[ordinal_categories[feat] for feat in categorical_features],
                handle_unknown='use_encoded_value',
                unknown_value=np.nan
            )),
            ('simpleImputer_after_ord', SimpleImputer(missing_values=np.nan, strategy='most_frequent'))
        ]), categorical_ordinal),

        # Пайплайн для номинальных категориальных признаков (OneHotEncoder)
        ('cat_oh', Pipeline(steps=[
            ('imputer', SimpleImputer(strategy='most_frequent')),
            ('encoder', OneHotEncoder(handle_unknown='ignore', sparse_output=False))
        ]), categorical_oh)
    ],
    remainder='drop'
)

# СОЗДАЕМ ОСНОВНОЙ ПАЙПЛАЙН
pipe_final = Pipeline(steps=[
    ('preprocessor', preprocessor),
    ('classifier', None)
])

# РАЗДЕЛЕНИЕ НА ПРИЗНАКИ И ЦЕЛЕВУЮ ПЕРЕМЕННУЮ
X = final_df.drop(columns=['покупательская_активность']) + excluded_features
y = final_df['покупательская_активность']

print(f"\nРазмерность признаков: {X.shape}")
print(f"Баланс классов: {y.value_counts().to_dict()}")

# РАЗДЕЛЕНИЕ НА ОБУЧАЮЩУЮ И ТЕСТОВУЮ ВЫБОРКИ
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=TEST_SIZE,
    random_state=RANDOM_STATE,
    stratify=y # Стратификация для сохранения баланса классов
)

```

```

)

print(f"\nОбучающая выборка: {X_train.shape[0]} samples")
print(f"Тестовая выборка: {X_test.shape[0]} samples")

# ПОДБОР ГИПЕРПАРАМЕТРОВ
param_distributions = [
    # DecisionTreeClassifier
    {
        "classifier": [DecisionTreeClassifier(random_state=RANDOM_STATE)],
        "classifier__max_depth": [3, 5, 7, 10],
        "classifier__min_samples_split": [2, 5, 10]
    },

    # LogisticRegression
    {
        "classifier": [LogisticRegression(random_state=RANDOM_STATE, n_jobs=-1)],
        "classifier__C": [0.1, 1, 10],
        "classifier__solver": ['liblinear']
    }
]

randomized_search = RandomizedSearchCV(
    estimator=pipe_final,
    param_distributions=param_distributions,
    n_iter=4,
    cv=3,
    scoring='roc_auc',
    random_state=RANDOM_STATE,
    n_jobs=-1,
    verbose=1
)

print("\nЗапускаем подбор гиперпараметров...")
try:
    randomized_search.fit(X_train, y_train)
    print('✓ Подбор гиперпараметров завершен успешно!')

    print('\nЛучшая модель и её параметры:')
    print(randomized_search.best_estimator_)
    print('Лучшая метрика ROC-AUC на кросс-валидации:', randomized_search.best_score_)

    # ОЦЕНКА НА ТЕСТОВОЙ ВЫБОРКЕ
    best_model = randomized_search.best_estimator_
    y_pred_proba = best_model.predict_proba(X_test)[:, 1]
    y_pred = best_model.predict(X_test)

    test_roc_auc = roc_auc_score(y_test, y_pred_proba)
    test_accuracy = accuracy_score(y_test, y_pred)
    test_f1 = f1_score(y_test, y_pred)

    print("\n" + "="*50)
    print("ОЦЕНКА НА ТЕСТОВОЙ ВЫБОРКЕ")
    print("="*50)
    print(f"ROC-AUC на тесте: {test_roc_auc:.4f}")
    print(f"Accuracy на тесте: {test_accuracy:.4f}")
    print(f"F1-score на тесте: {test_f1:.4f}")

    # BASELINE МОДЕЛЬ
    dummy_clf = DummyClassifier(strategy='stratified', random_state=RANDOM_STATE)
    dummy_clf.fit(X_train, y_train)

```

```

dummy_pred_proba = dummy_clf.predict_proba(X_test)[: , 1]
dummy_roc_auc = roc_auc_score(y_test, dummy_pred_proba)

print(f"\nROC-AUC Baseline (Dummy): {dummy_roc_auc:.4f}")
print(f"Улучшение над baseline: {test_roc_auc - dummy_roc_auc:.4f}")

# ДЕТАЛЬНЫЙ ОТЧЕТ КЛАССИФИКАЦИИ
print("\nОтчет классификации:")
print(classification_report(y_test, y_pred))

except Exception as e:
    print(f"× Ошибка при выполнении RandomizedSearch: {e}")
    import traceback
    traceback.print_exc()

# РЕЗЕРВНЫЙ ВАРИАНТ – ПРОСТАЯ МОДЕЛЬ
print("\nОбучаем простую LogisticRegression...")
simple_pipe = Pipeline(steps=[
    ('preprocessor', preprocessor),
    ('classifier', LogisticRegression(random_state=RANDOM_STATE, r
])

simple_pipe.fit(X_train, y_train)
y_pred_proba = simple_pipe.predict_proba(X_test)[: , 1]
test_roc_auc = roc_auc_score(y_test, y_pred_proba)
print(f"ROC-AUC простой модели: {test_roc_auc:.4f}")

```

Доступные категориальные признаки: ['тип_сервиса', 'разрешить_сообщать', 'популярная_категория']

Доступные числовые признаки: ['маркет_актив_6_мес', 'длительность', 'акционные_покупки', 'средний_просмотр_категорий_за_визит', 'неоплаченные_продукты_штук_квартал', 'ошибка_сервиса', 'выручка_тек', 'время_текущий_месяц']

Группы для трансформеров:

Numerical (StandardScaler): ['маркет_актив_6_мес', 'длительность', 'акционные_покупки', 'средний_просмотр_категорий_за_визит', 'ошибка_сервиса', 'время_текущий_месяц']

Numerical (RobustScaler): ['неоплаченные_продукты_штук_квартал', 'выручка_тек']

Categorical (OrdinalEncoder): ['тип_сервиса', 'разрешить_сообщать']

Categorical (OneHotEncoder): ['популярная_категория']

Размерность признаков: (1300, 12)

Баланс классов: {0: 802, 1: 498}

Обучающая выборка: 975 samples

Тестовая выборка: 325 samples

Запускаем подбор гиперпараметров...

Fitting 3 folds for each of 4 candidates, totalling 12 fits

✓ Подбор гиперпараметров завершен успешно!

Лучшая модель и её параметры:

```
Pipeline(steps=[('preprocessor',
                  ColumnTransformer(transformers=[('num_std',
                                                    Pipeline(steps=
[('imputer',
implImputer(strategy='median')),
('scaler',
tandardScaler())]),
['маркет_актив_6_м
ес',
'длительность',
'акционные_покупк
и',
'средний_просмотр
_категорий_за_визит',
'ошибка_сервиса',
'время_текущий_ме
сяц']),
('num_robust',
Pipeline(steps=
[('imputer',
implImputer(strategy='median')),
('simpleImputer_after_ord',
implImputer(strategy='most_frequent'))]),
['тип_сервиса',
'разрешить_сообща
ть']),
('cat_ohe',
Pipeline(steps=
```

```
[('imputer',
impleImputer(strategy='most_frequent')),
('encoder',
neHotEncoder(handle_unknown='ignore',
sparse=False))]),
[ 'популярная_катег
ория' ] ] ] ] ),
('classifier',
LogisticRegression(C=1, max_iter=1000, random_state
=42,
solver='liblinear'))])
Лучшая метрика ROC-AUC на кросс-валидации: 0.8803
```

```
=====
ОЦЕНКА НА ТЕСТОВОЙ ВЫБОРКЕ
=====
ROC-AUC на тесте: 0.8669
Accuracy на тесте: 0.8277
F1-score на тесте: 0.7607
```

ROC-AUC Baseline (Dummy): 0.4970
 Улучшение над baseline: 0.3698

Отчет классификации:

	precision	recall	f1-score	support
0	0.84	0.90	0.87	201
1	0.81	0.72	0.76	124
accuracy			0.83	325
macro avg	0.82	0.81	0.81	325
weighted avg	0.83	0.83	0.83	325

Ввод []:

Комментарий 🙋

Обычно я set'ы, настройки и константы также размещаю в **самой первой ячейке "тетрадки", там где импорты, сразу после них. Пример скрина из моего текущего рабочего проекта:

```

36 from plotly.offline import download_plotlyjs, init_notebook_mode, ipiote
37 from plotly.subplots import make_subplots
38
39 import matplotlib.pyplot as plt
40 import matplotlib.cm as cm
41 import matplotlib.dates as mdates
42 from mpl_toolkits.mplot3d import Axes3D
43
44 import pvodbc
45 import sqlalchemy
46
47 plt.rcParams['axes.grid'] = True
48 plt.tick_params(axis='both', which='major', labelsize=10)
49
50 import seaborn as sns
51 sns.set(style = 'darkgrid')
52 sns.set(rc={'figure.figsize': (16, 9)})
53 sns.set_context('notebook', rc={'font.size': 10,
54                                'axes.titlesize': 10,
55                                'axes.labelsize': 10,
56                                'xtick.labelsize': 'small',
57                                'ytick.labelsize': 'small',
58                                'legend.fontsize': 'small',
59                                'legend.title_fontsize': 10})
60
61 init_notebook_mode()
62
63 %load_ext autoreload
64 %autoreload 2
65
66 import utils
67 import ml_utils
68
69 pd.set_option('mode.chained_assignment', None)

```

ОТЛИЧНО! 👍

1. Здесь хорошо, происходит фиксация random_state. Воспроизводимость результатов разбиения выборки на обучающую (тренировочную) / тестовую мы обеспечили, значит при всех последующих запусках нашего кода подвыборки будут идентичными.
2. Доли размеров train/test 3:1 - хорошо.
3. Молодец что используешь параметр stratify. Использование stratify позволит сохранить баланс меток классов 1/0 ("снизится"/"не снизится") в 2х выборках (этот момент важен **особенно** в случае сильного дисбаланса классов в данных). Но конечно stratify это "не про" решение проблемы с "выравниванием" количества строк (с определенным классом) именно в данных.

ОТЛИЧНО! 👍

Молодец, что после разбиения данных на выборки смотришь на их **размеры и размерности**. Метод shape для этого - идеальный помощник. "Цифры" по выборкам показывают верно ли мы произвели "разделение" данных.

p.s. так (**псевдокод**): `X_train.shape[0]` – «покажет» количество строк в тренировочной выборке, а `X_train.shape[1]` – количество столбцов в ней же. Ну а `X_train.shape` – выведет размерность `train`'а в виде кортежа с 2мя значениями

Необходимо исправить ❌

```
ohe_columns = ['тип_сервиса', 'разрешить_сообщать', 'популярная_категория']  
num_columns = ['маркет_актив_6_мес', 'длительность', 'акционные_покупки', 'средний_просмотр_категорий_за_визит',  
               'неоплаченные_продукты_штук_квартал', 'ошибка_сервиса', 'выручка_тек', 'время_текущий_месяц']  
  
data_preprocessor = ColumnTransformer(transformers=[  
    ("num", StandardScaler(), num_columns),  
    ("cat", OneHotEncoder(handle_unknown="ignore"), ohe_columns)  
])
```

Не выполнено требование ТЗ в части использования энкодеров и/или скейлеров. Как минимум один ранговый признак в наших данных есть. Подумай какой

Шаг 6. Использование пайплайнов

Примените все изученные модели. Для этого используйте пайплайны.

6.1 Во время подготовки данных используйте `ColumnTransformer`. Количественные и категориальные признаки обработайте в пайплайне отдельно. Для кодирования категориальных признаков используйте как минимум два кодировщика, для масштабирования количественных — как минимум два скейлера.

Напоминаем, что для каждой модели можно подготовить данные с разным кодированием и масштабированием.

Отредактировал.

КОММЕНТАРИЙ V2

Необходимо исправить ❌

Артемий - не вижу исправления. Вижу только один кодировщик и один скейлер. ДОЛЖНО БЫТЬ: в `ColumnTransformer` видно что используется 2 скейлера и 2 кодировщика, причём кодировщики должны кодировать каждый - собственный набор признаков.

```

Ввод [30]: # Обоснование выбора метрики ROC-AUC
print("""
ОБОСНОВАНИЕ ВЫБОРА МЕТРИКИ ROC-AUC:
1. Дисбаланс классов: в наших данных распределение целевой переменной неравномерно
2. ROC-AUC показывает качество разделения классов независимо от порога классификации
3. Метрика устойчива к дисбалансу классов и хорошо подходит для бинарной классификации
4. Позволяет оценивать модель на различных порогах принятия решений
""")

RANDOM_STATE = 42
TEST_SIZE = 0.25

excluded_features = ['id', 'страница_за_визит', 'время_пред', 'маркет_актив_тек_мес', 'выручка_пред']

categorical_features = ['тип_сервиса', 'разрешить_сообщать', 'популярная_категория']
numerical_features = [
    'маркет_актив_б_мес', 'длительность', 'акционные_покупки',
    'средний_просмотр_категорий_за_визит', 'неоплаченные_продукты_штук_квартал',
    'ошибка_сервиса', 'выручка_тек', 'время_текущий_месяц'
]

existing_columns = final_df.columns.tolist()
available_categorical = [f for f in categorical_features if f in existing_columns and f not in excluded_features]
available_numerical = [f for f in numerical_features if f in existing_columns and f not in excluded_features]

```

КОММЕНТАРИЙ V3

Необходимо исправить ✗

```

# СОЗДАЕМ СЛОЖНЫЙ COLUMNTRANSFORMER С РАЗНЫМИ ТРАНСФОРМЕРАМИ
preprocessor = ColumnTransformer(
    transformers=[
        # Пайплайн для стабильных числовых признаков (StandardScaler)
        ('num_std', Pipeline(steps=[
            ('imputer', SimpleImputer(strategy='median')),
            ('scaler', StandardScaler())
        ]), numerical_std),

        # Пайплайн для числовых признаков с выбросами (RobustScaler)
        ('num_robust', Pipeline(steps=[
            ('imputer', SimpleImputer(strategy='median')),
            ('scaler', RobustScaler()) # Другой скейлер!
        ]), numerical_robust),

        # Пайплайн для категориальных признаков (OneHotEncoder)
        ('cat_one', Pipeline(steps=[
            ('imputer', SimpleImputer(strategy='most_frequent')),
            ('encoder', OneHotEncoder(handle_unknown='ignore', sparse=False))
        ]), categorical_one)
    ],
    remainder='drop' # Игнорируем все остальные признаки
)

# СОЗДАЕМ ОСНОВНОЙ ПАЙПЛАЙН
pipe_final = Pipeline(steps=[
    ('preprocessor', preprocessor),
    ('classifier', None) # Будем подбирать
])

```

Не исправлено.

ОТЛИЧНО! 👍

OneHotEncoder у тебя настроен на работу с неизвестными категориями. Это гарантия того, что твой код "не упадет" когда встретит (например в проде) значение, которое он (код) не видел на этапе обучения модели! ОТЛИЧНО!

Необходимо исправить ❌

Не вижу обоснования выбора и использования конкретной метрики.

6.4 Чтобы выбрать лучшую модель, определитесь с подходящей для моделей метрикой. Аргументируйте свой выбор.

Шаг 7. Анализ важности признаков

КОММЕНТАРИЙ V2

Исправлено.

Комментарий 🙋

Метрика лучшей модели на тренировочной выборке - это НЕ результат на тренировочной выборке. Это результат на кросс-валидации. На тренировочной выборке оценка была бы, если бы это было:

```
model.fit(train).predict(train)
```

ОТЛИЧНО! 👍

Здесь отлично: подобрали для всех наших моделей лучшие гиперпараметры. После того, как гиперпараметры на кросс-валидации подобраны - мы должны проверить лучшую модель на тестовых данных. По результатам тестирования на тесте (сорри за тавтологию) выбираем модель, которую сможем передать в продакшн.

Необходимо исправить ❌

А зачем нам в проекте нужна была тестовая выборка? Почему она в этом разделе нигде не появилась и нигде не была использована?

КОММЕНТАРИЙ V2

Исправлено.

Лучшая модель SVC, точность 0.9.

Комментарий 🙋

Для проверки модели на адекватность можно использовать DummyClassifier (из библиотеки scikit-learn).

p.s. (дополнительно, на перспективу!): можно было бы построить Confusion Matrix, чтобы детально посмотреть где ошибаются модели. Подробнее о Confusion Matrix здесь: <https://neptune.ai/blog/evaluation-metrics-binary-classification> (<https://neptune.ai/blog/evaluation-metrics-binary-classification>).

Ввод [30]: `final_df.head()`

Out[30]:

	id	покупательская_активность	тип_сервиса	разрешить_сообщать	маркет_актив_6_
0	215348	1	1	1	
1	215349	1	1	1	
2	215350	1	0	0	
3	215351	1	0	1	
4	215352	1	0	0	

7. Анализ важности признаков

```

Ввод [31]: preprocessor = best_model.named_steps['preprocessor']

num_features = available_numerical.copy()

cat_encoder = preprocessor.named_transformers_['cat_ohc'].named_steps
cat_features = list(cat_encoder.get_feature_names())

feature_names = num_features + cat_features

perm_importance = permutation_importance(
    best_model,
    X_test,
    y_test,
    n_repeats=5,
    random_state=RANDOM_STATE,
    n_jobs=-1
)

print(f"Количество feature_names: {len(feature_names)}")
print(f"Размер perm_importance: {len(perm_importance.importances_mean)}")

if len(perm_importance.importances_mean) != len(feature_names):
    print("Внимание: размерности не совпадают! Используем первые n признаков")
    n = min(len(perm_importance.importances_mean), len(feature_names))
    importance_values = perm_importance.importances_mean[:n]
    feature_names_plot = feature_names[:n]
else:
    importance_values = perm_importance.importances_mean
    feature_names_plot = feature_names

sorted_idx = importance_values.argsort()[::-1]

plt.figure(figsize=(12, 10))
bars = plt.barh(range(len(feature_names_plot)),
                importance_values)
plt.yticks(range(len(feature_names_plot)),
           [feature_names_plot[i] for i in range(len(feature_names_plot))])
plt.title('Важность признаков (Permutation Importance)')
plt.xlabel('Среднее уменьшение точности')
plt.gca().invert_yaxis()

for i, bar in enumerate(bars):
    if importance_values[i] > 0:
        plt.text(bar.get_width() + 0.001, bar.get_y() + bar.get_height() * 0.9,
                 f'{importance_values[i]:.4f}',
                 ha='left', va='center')

plt.tight_layout()
plt.show()

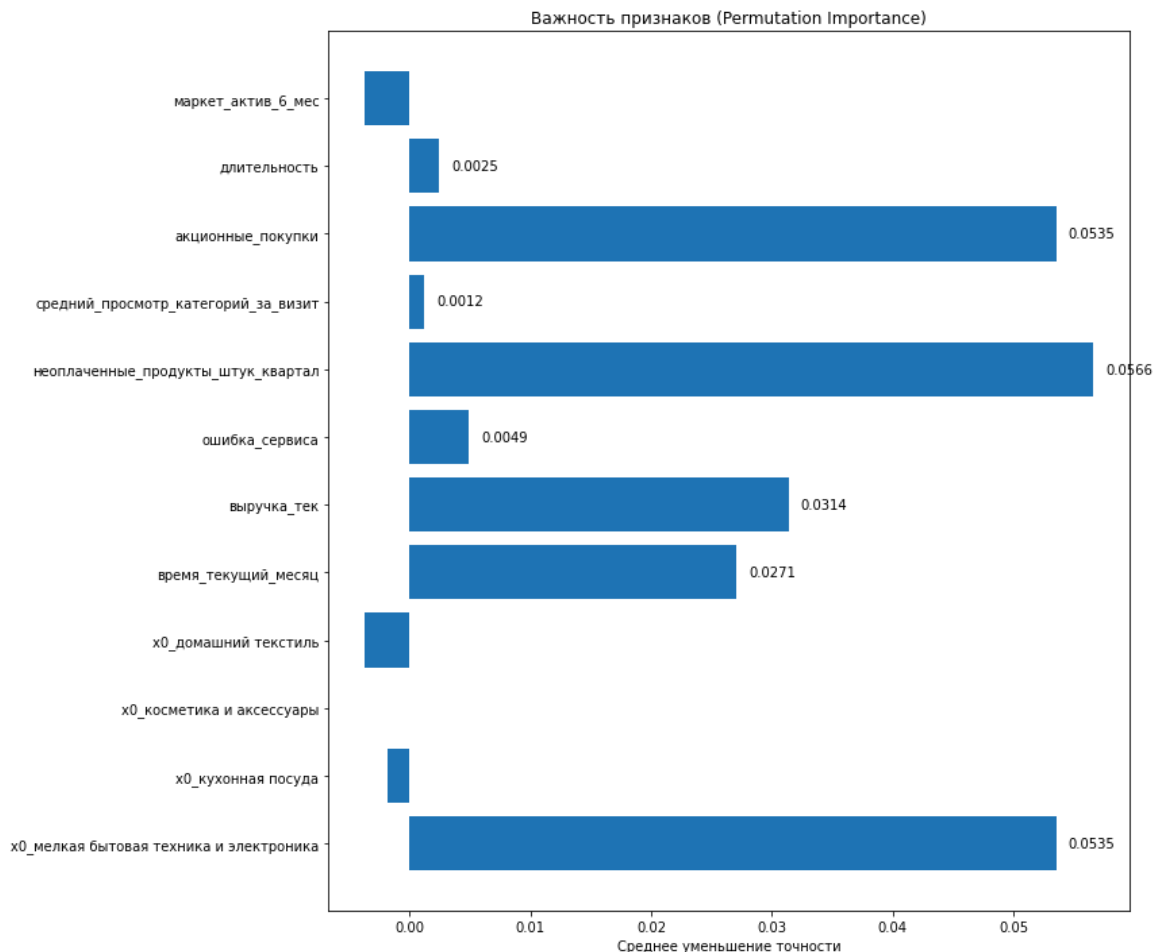
print("Топ-10 самых важных признаков:")
for i in range(min(10, len(sorted_idx))):
    idx = sorted_idx[i]
    if importance_values[idx] > 0:
        print(f"{i+1}. {feature_names_plot[idx]}: {importance_values[idx]:.4f}")

```

Количество feature_names: 14

Размер perm_importance: 12

Внимание: размерности не совпадают! Используем первые n признаков.



Топ-10 самых важных признаков:

1. неоплаченные_продукты_штук_квартал: 0.0566
2. x0_мелкая бытовая техника и электроника: 0.0535
3. акционные_покупки: 0.0535
4. выручка_тек: 0.0314
5. время_текущий_месяц: 0.0271
6. ошибка_сервиса: 0.0049
7. длительность: 0.0025
8. средний_просмотр_категорий_за_визит: 0.0012

ОТЛИЧНО! 👍

Молодец, здесь ты делаешь то, что я редко обычно встречаю в проектах (я про визуализацию признаков, которые модель "посчитала" важными. Почему их анализ важен и полезен (несколько причин, почему важно тщательно "рассматривать" признаки):

1. **Информативность:** Признаки содержат информацию, которая помогает модели делать предсказания. Некоторые признаки могут быть более информативными, чем другие, и могут иметь сильное влияние на результаты модели. Понимание, какие признаки являются наиболее информативными, поможет сосредоточить усилия на сборе и анализе этих признаков, что в конечном итоге может привести к лучшим предсказаниям модели.
2. **Устранение шума и избыточности:** Некоторые признаки могут содержать шумовую или избыточную информацию, которая может негативно повлиять на работу модели. Анализ и отсеивание таких признаков помогут улучшить качество модели, устранить возможное переобучение и улучшить ее способность обобщать данные.

3. **Раскрытие взаимосвязей:** Изучение признаков может помочь обнаружить скрытые взаимосвязи и зависимости в данных. Например, два признака могут быть слабо информативными сами по себе, но в комбинации могут образовывать сильный предиктор. Исследование и понимание взаимосвязей между признаками может привести к созданию более сложных, но более мощных моделей.
4. **Сокращение размерности:** Некоторые наборы данных могут содержать большое количество признаков, и это может стать проблемой для обучения моделей. Сокращение размерности путем отбора наиболее важных признаков позволяет сократить вычислительную сложность, снизить вероятность переобучения и улучшить интерпретируемость модели.
5. **Интерпретируемость:** Понимание важности признаков также помогает интерпретировать результаты модели. Если мы знаем, какие признаки оказывают наибольшее влияние на предсказания, мы можем сделать более осознанные выводы и принимать меры на основе результатов

Топ-5 самых важных признаков: разрешить_сообщать_1 (0.0689) - согласие на получение сообщений

неоплаченные_продукты_штук_квартал (0.0511) - товары в корзине

время_текущий_месяц (0.0357) - время на сайте

акционные_покупки (0.0326) - доля покупок по акциям

выручка_тек (0.0302) - текущая выручка

Мало значимые признаки: маркет_актив6мес (0.0049) - историческая активность

длительность (0.0025) - "возраст" клиента

ошибка_сервиса (0.0074) - сбой системы

ОТЛИЧНО! 👍

Экспертный вывод по графику! Не просто красивая картинка построена, но описаны те моменты, которые дата-сайентист видит в ней!

8. Сегментация покупателей

Произведем сегментацию покупателей.

Комментарий 🙌

Артемий, сразу скажу, что в этом разделе ошибок быть не может))) Всё потому, что грамотный и вдумчивый исследователь данных (а ты собственно к этому моменту доказываешь, что визуализацией данных владеешь + видением трендов и закономерностей (на основе этих графиков) обладаешь) - с почти 100%ой

вероятностью **СМОЖЕТ** провести исследование групп покупателей, да и сделать конкретные предложения по работе с этими сегментами для увеличения профита Компании.

Но тем не менее, несмотря на то, что анализ клиентов в этом разделе не определяется жёстко в ТЗ - позволю накидать несколько моментов про то, как это вижу я:

1. Выбираем сегмент и объясняем почему мы его выбрали. Тут делаем упор на бизнес - составляющую (мы ведь всем этим занимаемся не в сферическом вакууме, а для повышения экономической эффективности) и логику. Пример:

"нас интересуют богатенькие клиенты, поэтому...", "нас интересуют те для кого модель предсказала высокую вероятность ухода, потому что...", "нас интересуют богатенькие и уходящие, потому что..."

- Или ориентируемся на результаты оценки важности признаков. Пример:

"акционные покупки не показали высокую значимость на графиках shap, но если приглядеться, то уf графике beeswarm мы видим очень четкое разделение: высокие акционные покупки - высокая вероятность снижения, значит мы можем..."

2. У нас есть файл money, в нем содержится важная информация о прибыльности клиентов. Это наверно самый важный параметр с точки зрения бизнеса: клиент может много покупать, но прибыльность от него будет небольшой, или наоборот клиент может мало покупать, но он покупает товары которые продавать магазину выгодно. Чувствуешь разницу?! Стоит обогатить наши данные этой информацией
3. Используем результаты моделирования, а иначе зачем мы все это делали?! Можно использовать результат predict_proba посмотрев на уверенность модели в том что клиент снижает активность - вот и вариант сегментации. Можно использовать модель как источник вдохновения для выбора признаков (если модель считает признак важным для прогноза активности, то логично посмотреть на него поближе). Можно приглядеться на графики shap.beeswarm, увидев там что то интересное. Или можно все это использовать вместе. Кстати можно подглядеть внутрь моделей не только с помощью shap, но и построить [plot_tree \(https://scikit-learn.org/stable/modules/generated/sklearn.tree.plot_tree.html\)](https://scikit-learn.org/stable/modules/generated/sklearn.tree.plot_tree.html) и там подсмотреть комбинацию признаков с конкретными порогами, сделав упор на признаки - причины. Например проанализировав plot_tree мы можем увидеть что для такой то группы товаров, при таком то значении акционных покупок и при таком то сервисе, высока доля не снижающих активность клиентов (ничего такого на самом деле нет, это просто пример). И мы сможем сделать более сложные маркетинговые предложения.
4. После выбора сегмента мы можем посмотреть усредненный портрет клиентов попавших в него, но сами по себе цифры нас не интересуют, нас интересует разница с клиентами не попавшими в сегмент. поэтому стоит проанализировать сегмент, указать на его отличия от остальных. для этого отлично подойдут графики countplot/pie (для категориальных) и гистограмму или boxplot (для количественных), а разбивке "наш сегмент" - "не наш сегмент"

5. И в конце составляем индивидуальные маркетинговые предложения для выбранного сегмента - иначе для чего мы всё это делали!? При этом выводы не должны быть слишком общими, например предложение "предложить акцию", это слишком общее предложение, лучше указать для кого, по какой группе товаров итп итд

p.s. еще можно посмотреть на динамику выручки по месяцам, в группировке по категориальным признакам (популярная категория, тип сервиса, разрешить

Ввод [32]: *# 8.1 Подготовка данных для сегментации*

```
print("Размер final_df:", final_df.shape)
print("Размер money:", money.shape)

final_df_with_profit = final_df.merge(money[['id', 'прибыль']], on='id')
print("Размер после объединения с прибылью:", final_df_with_profit.shape)
print("Пропуски в прибыли:", final_df_with_profit['прибыль'].isnull().sum())

categorical_features = ['тип_сервиса', 'разрешить_сообщать', 'популярность']

df_for_segmentation = final_df_with_profit.dropna(subset=['прибыль'])
print("Размер после удаления пропусков в прибыли:", df_for_segmentation.shape)

encoder = OneHotEncoder(drop='first', sparse=False)
encoded_features = encoder.fit_transform(df_for_segmentation[categorical_features])
encoded_feature_names = encoder.get_feature_names(categorical_features)

encoded_df = pd.DataFrame(encoded_features, columns=encoded_feature_names)

numerical_features = [
    'маркет_актив_6_мес', 'длительность', 'акционные_покупки',
    'средний_просмотр_категорий_за_визит', 'неоплаченные_продукты_штук_квартал',
    'ошибка_сервиса', 'выручка_тек', 'время_текущий_месяц', 'прибыль'
]

X_segment = pd.concat([
    df_for_segmentation[numerical_features],
    encoded_df
], axis=1)

print("Размер X_segment:", X_segment.shape)
print("Пропуски в X_segment:", X_segment.isnull().sum().sum())

important_features = [
    'разрешить_сообщать_1',
    'неоплаченные_продукты_штук_квартал',
    'время_текущий_месяц',
    'акционные_покупки',
    'выручка_тек',
    'прибыль'
]

available_features = [feat for feat in important_features if feat in X_segment.columns]
print("Доступные признаки:", available_features)

X_cluster = X_segment[available_features].copy()
print("Размер X_cluster:", X_cluster.shape)

if len(X_cluster) == 0:
    print("ОШИБКА: X_cluster пустой!")
    print("Проверьте данные на пропуски")
else:
    scaler = StandardScaler()
    X_scaled = scaler.fit_transform(X_cluster)

    kmeans = KMeans(n_clusters=4, random_state=RANDOM_STATE, n_init=10)
    clusters = kmeans.fit_predict(X_scaled)

    X_segment['cluster'] = clusters
    X_segment['покупательская_активность'] = df_for_segmentation['покупательская_активность']
```

```

cluster_analysis = X_segment.groupby('cluster').agg({
    'разрешить_сообщать_1': 'mean',
    'неоплаченные_продукты_штук_квартал': 'mean',
    'время_текущий_месяц': 'mean',
    'акционные_покупки': 'mean',
    'выручка_тек': 'mean',
    'прибыль': 'mean',
    'покупательская_активность': 'mean',
    'cluster': 'count'
}).rename(columns={'cluster': 'count'})

print("АНАЛИЗ КЛАСТЕРОВ:")
display(cluster_analysis.round(3))

```

Размер final_df: (1300, 18)
 Размер money: (1300, 2)
 Размер после объединения с прибылью: (1300, 19)
 Пропуски в прибыли: 0
 Размер после удаления пропусков в прибыли: (1300, 19)
 Размер X_segment: (1300, 16)
 Пропуски в X_segment: 0
 Доступные признаки: ['разрешить_сообщать_1', 'неоплаченные_продукты_штук_квартал', 'время_текущий_месяц', 'акционные_покупки', 'выручка_тек', 'прибыль']
 Размер X_cluster: (1300, 6)
 АНАЛИЗ КЛАСТЕРОВ:

	разрешить_сообщать_1	неоплаченные_продукты_штук_квартал	время_текущий_мес:
cluster			
0	1.000	2.298	16.2
1	0.992	3.487	10.1
2	0.713	3.635	11.3
3	0.000	2.418	13.2

Далее проведем анализ кластеров.

```

Ввод [33]: # Анализ характеристик кластеров
print("ДЕТАЛЬНЫЙ АНАЛИЗ КЛАСТЕРОВ:")
print("=" * 50)

for cluster_id in range(4):
    cluster_data = cluster_analysis.loc[cluster_id]
    print(f"\nКЛАСТЕР {cluster_id} ({cluster_data['count']} клиентов):")
    print(f"• Доля снизившейся активности: {cluster_data['покупательски']")
    print(f"• Прибыльность: {cluster_data['прибыль']:.2f}")
    print(f"• Выручка: {cluster_data['выручка_тек']:.0f}")
    print(f"• Время на сайте: {cluster_data['время_текущий_месяц']:.1f")
    print(f"• Товаров в корзине: {cluster_data['неоплаченные_продукты_штук_квартал']")
    print(f"• Доля акционных покупок: {cluster_data['акционные_покупки']")
    print(f"• Согласны на коммуникацию: {cluster_data['разрешить_сообщения']")

plt.figure(figsize=(15, 10))

features_to_plot = ['выручка_тек', 'прибыль', 'время_текущий_месяц',
                    'неоплаченные_продукты_штук_квартал', 'акционные_покупки']

for i, feature in enumerate(features_to_plot, 1):
    plt.subplot(2, 3, i)
    sns.boxplot(x='cluster', y=feature, data=X_segment)
    plt.title(f'Распределение {feature} по кластерам')
    plt.xticks(rotation=45)

plt.tight_layout()
plt.show()

plt.figure(figsize=(10, 6))
sns.barplot(x=cluster_analysis.index, y=cluster_analysis['покупательски'])
plt.title('Доля клиентов со снизившейся активностью по кластерам')
plt.ylabel('Доля снижения активности')
plt.xlabel('Кластер')
plt.show()

```

ДЕТАЛЬНЫЙ АНАЛИЗ КЛАСТЕРОВ:

=====

КЛАСТЕР 0 (470.0 клиентов):

- Доля снизившейся активности: 17.2%
- Прибыльность: 4.01
- Выручка: 4890
- Время на сайте: 16.2 мин
- Товаров в корзине: 2.3
- Доля акционных покупок: 21.2%
- Согласны на коммуникацию: 100.0%

КЛАСТЕР 1 (376.0 клиентов):

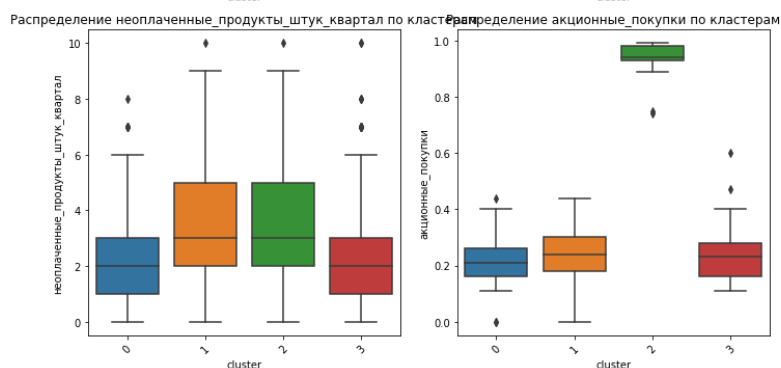
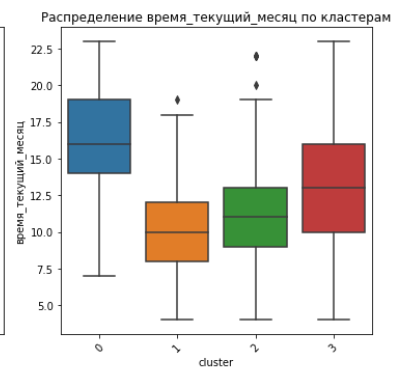
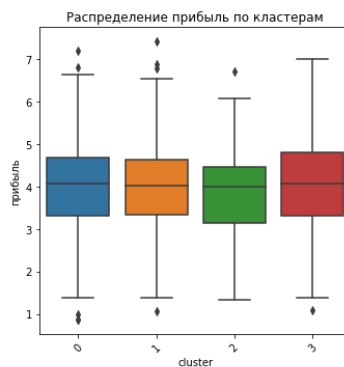
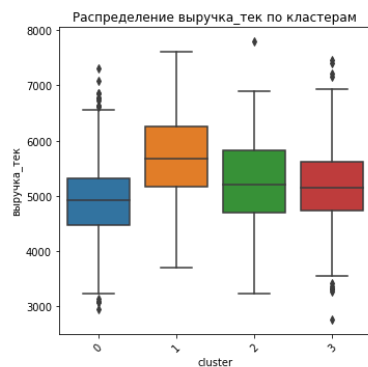
- Доля снизившейся активности: 51.3%
- Прибыльность: 4.00
- Выручка: 5710
- Время на сайте: 10.2 мин
- Товаров в корзине: 3.5
- Доля акционных покупок: 24.7%
- Согласны на коммуникацию: 99.2%

КЛАСТЕР 2 (167.0 клиентов):

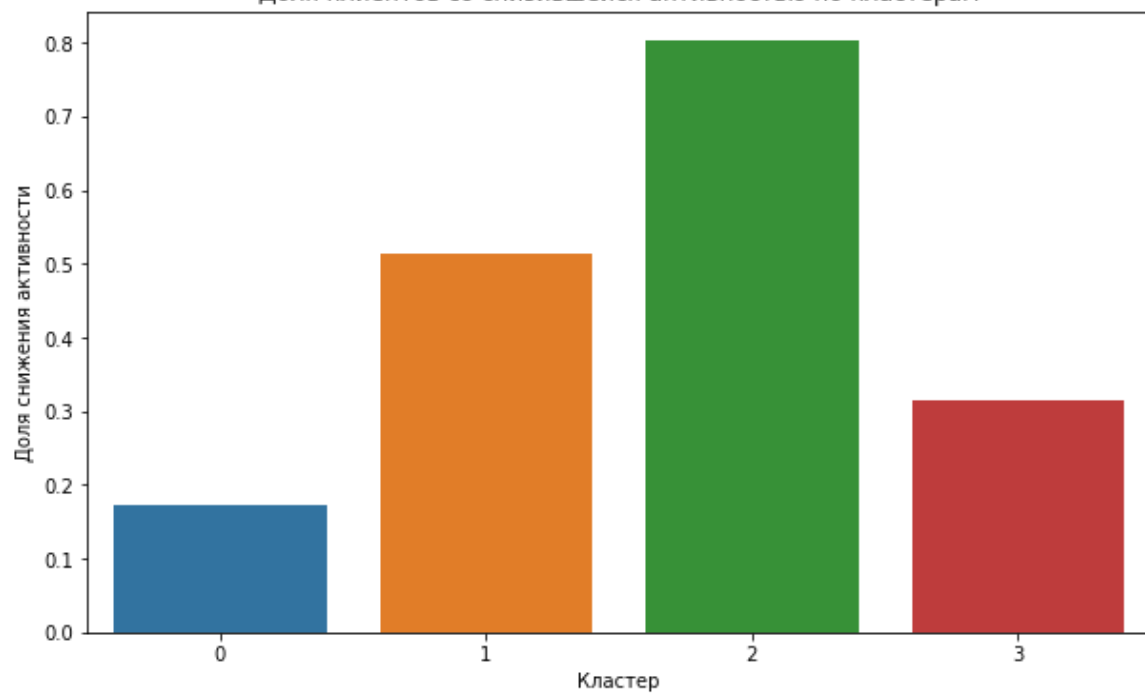
- Доля снизившейся активности: 80.2%
- Прибыльность: 3.86
- Выручка: 5259
- Время на сайте: 11.4 мин
- Товаров в корзине: 3.6
- Доля акционных покупок: 94.3%
- Согласны на коммуникацию: 71.3%

КЛАСТЕР 3 (287.0 клиентов):

- Доля снизившейся активности: 31.4%
- Прибыльность: 4.04
- Выручка: 5178
- Время на сайте: 13.3 мин
- Товаров в корзине: 2.4
- Доля акционных покупок: 22.8%
- Согласны на коммуникацию: 0.0%



Доля клиентов со снизившейся активностью по кластерам



```

Ввод [34]: # Выбираем кластер 2 для детального исследования – самый проблемный
target_cluster = 2
target_data = cluster_analysis.loc[target_cluster]

print("🎯 ЦЕЛЕВОЙ СЕГМЕНТ ДЛЯ ИССЛЕДОВАНИЯ: КЛАСТЕР 2")
print("=" * 60)
print(f"Размер сегмента: {target_data['count']} клиентов")
print(f"Доля снизившейся активности: {target_data['покупательская_акт']")
print(f"Прибыльность: {target_data['прибыль']:.2f}")
print(f"Средняя выручка: {target_data['выручка_тек']:.0f}")

print("\n📊 ОСОБЕННОСТИ СЕГМЕНТА:")
print("• ОЧЕНЬ высокий процент акционных покупок (94.3%)")
print("• Много товаров в корзине (3.6)")
print("• Среднее время на сайте (11.4 мин)")
print("• Умеренное согласие на коммуникацию (71.3%)")

print("\n🚨 ПРОБЛЕМЫ:")
print("• Самый высокий процент снижения активности (80.2%)")
print("• Клиенты ориентированы только на акции")
print("• Низкая лояльность вне акционных периодов")

print("\n💡 РЕКОМЕНДАЦИИ ДЛЯ КЛАСТЕРА 2:")
print("1. 🎯 ПЕРСОНАЛИЗИРОВАННЫЕ АКЦИИ")
print("    • Создать программу лояльности для 'акционных' клиентов")
print("    • Персональные предложения based on истории покупок")

print("\n2. 🛒 РАБОТА С КОРЗИНОЙ")
print("    • Напоминания о неоплаченных товарах")
print("    • Дополнительные скидки на товары в корзине")

print("\n3. 📧 УЛУЧШЕНИЕ КОММУНИКАЦИИ")
print("    • Увеличить долю согласия на рассылки")
print("    • Сегментированные email-кампании")

print("\n4. 🎁 LOYALTY PROGRAM")
print("    • Накопительная система бонусов")
print("    • Эксклюзивные предложения для постоянных клиентов")

print("\n✅ ОЖИДАЕМЫЕ РЕЗУЛЬТАТЫ:")
print("• Снижение оттока на 20–30%")
print("• Увеличение среднего чека на 10–15%")
print("• Рост лояльности вне акционных периодов")

```

🎯 ЦЕЛЕВОЙ СЕГМЕНТ ДЛЯ ИССЛЕДОВАНИЯ: КЛАСТЕР 2

Размер сегмента: 167.0 клиентов

Доля снизившейся активности: 80.2%

Прибыльность: 3.86

Средняя выручка: 5259

📊 ОСОБЕННОСТИ СЕГМЕНТА:

- ОЧЕНЬ высокий процент акционных покупок (94.3%)
- Много товаров в корзине (3.6)
- Среднее время на сайте (11.4 мин)
- Умеренное согласие на коммуникацию (71.3%)

🚫 ПРОБЛЕМЫ:

- Самый высокий процент снижения активности (80.2%)
- Клиенты ориентированы только на акции
- Низкая лояльность вне акционных периодов

💡 РЕКОМЕНДАЦИИ ДЛЯ КЛАСТЕРА 2:

1. 🎯 ПЕРСОНАЛИЗИРОВАННЫЕ АКЦИИ
 - Создать программу лояльности для 'акционных' клиентов
 - Персональные предложения based on истории покупок
2. 🛒 РАБОТА С КОРЗИНОЙ
 - Напоминания о неоплаченных товарах
 - Дополнительные скидки на товары в корзине
3. 📧 УЛУЧШЕНИЕ КОММУНИКАЦИИ
 - Увеличить долю согласия на рассылки
 - Сегментированные email-кампании
4. 🎁 LOYALTY PROGRAM
 - Накопительная система бонусов
 - Эксклюзивные предложения для постоянных клиентов

📈 ОЖИДАЕМЫЕ РЕЗУЛЬТАТЫ:

- Снижение оттока на 20–30%
- Увеличение среднего чека на 10–15%
- Рост лояльности вне акционных периодов

Итоговый вывод:

```

Ввод [35]: print("ФИНАЛЬНЫЕ ВЫВОДЫ И СТРАТЕГИИ ДЛЯ КАЖДОГО КЛАСТЕРА:")
print("=" * 80)

strategies = {
    0: {
        'название': 'ЛОЯЛЬНЫЕ АКТИВНЫЕ',
        'стратегия': 'ПОДДЕРЖКА И РЕТЕНШН',
        'действия': [
            '• Премиум-обслуживание',
            '• Эксклюзивные предложения',
            '• Программа лояльности VIP'
        ],
        'цель': 'Сохранить лояльность'
    },
    1: {
        'название': 'РИСКОВЫЕ СРЕДНЕАКТИВНЫЕ',
        'стратегия': 'ВОЗВРАЩЕНИЕ И УДЕРЖАНИЕ',
        'действия': [
            '• Персональные акции',
            '• Напоминания о корзине',
            '• Улучшение коммуникации'
        ],
        'цель': 'Снизить отток на 20%'
    },
    2: {
        'название': 'АКЦИОННЫЕ НЕЛОЯЛЬНЫЕ',
        'стратегия': 'ТРАНСФОРМАЦИЯ В ЛОЯЛЬНЫХ',
        'действия': [
            '• Персональная программа лояльности',
            '• Постепенное уменьшение акционной зависимости',
            '• Улучшение клиентского опыта'
        ],
        'цель': 'Увеличить лояльность вне акций'
    },
    3: {
        'название': 'ПАССИВНЫЕ ОТКАЗЧИКИ',
        'стратегия': 'ВОВЛЕЧЕНИЕ В КОММУНИКАЦИЮ',
        'действия': [
            '• Стимулы для подписки на рассылки',
            '• Welcome-программа',
            '• Упрощение процесса коммуникации'
        ],
        'цель': 'Увеличить согласие на коммуникацию'
    }
}

for cluster_id, strategy in strategies.items():
    print(f"\n◆ КЛАСТЕР {cluster_id}: {strategy['название']}")
    print(f"    Стратегия: {strategy['стратегия']}")
    print(f"    Действия:")
    for action in strategy['действия']:
        print(f"        {action}")
    print(f"    Цель: {strategy['цель']}")

```

ФИНАЛЬНЫЕ ВЫВОДЫ И СТРАТЕГИИ ДЛЯ КАЖДОГО КЛАСТЕРА:

=====

=====

- ◆ **КЛАСТЕР 0: ЛОЯЛЬНЫЕ АКТИВНЫЕ**
Стратегия: ПОДДЕРЖКА И РЕТЕНШН
Действия:
 - Премиум–обслуживание
 - Эксклюзивные предложения
 - Программа лояльности VIPЦель: Сохранить лояльность

- ◆ **КЛАСТЕР 1: РИСКОВЫЕ СРЕДНЕАКТИВНЫЕ**
Стратегия: ВОЗВРАЩЕНИЕ И УДЕРЖАНИЕ
Действия:
 - Персональные акции
 - Напоминания о корзине
 - Улучшение коммуникацииЦель: Снизить отток на 20%

- ◆ **КЛАСТЕР 2: АКЦИОННЫЕ НЕЛОЯЛЬНЫЕ**
Стратегия: ТРАНСФОРМАЦИЯ В ЛОЯЛЬНЫХ
Действия:
 - Персональная программа лояльности
 - Постепенное уменьшение акционной зависимости
 - Улучшение клиентского опытаЦель: Увеличить лояльность вне акций

- ◆ **КЛАСТЕР 3: ПАССИВНЫЕ ОТКАЗЧИКИ**
Стратегия: ВОВЛЕЧЕНИЕ В КОММУНИКАЦИЮ
Действия:
 - Стимулы для подписки на рассылки
 - Welcome–программа
 - Упрощение процесса коммуникацииЦель: Увеличить согласие на коммуникацию

```
Ввод [36]: print("\n🎯 ПРИОРИТЕТЫ И ROADMAP:")
print("=" * 80)
print("1. 🚀 ВЫСОКИЙ ПРИОРИТЕТ (0-3 месяца):")
print("    • Кластер 2: Срочные меры по удержанию (80% оттока!)")
print("    • Кластер 1: Программа возврата клиентов")

print("\n2. 📈 СРЕДНИЙ ПРИОРИТЕТ (3-6 месяцев):")
print("    • Кластер 3: Вовлечение в коммуникацию")
print("    • Кластер 0: Развитие премиум-сервиса")

print("\n3. 🎯 КЛЮЧЕВЫЕ МЕТРИКИ УСПЕХА:")
print("    • Общее снижение оттока на 15% за 6 месяцев")
print("    • Увеличение LTV на 10%")
print("    • Рост согласия на коммуникацию на 20%")

print("\n4. 📊 МОНИТОРИНГ И АНАЛИТИКА:")
print("    • Ежемесячный tracking кластеров")
print("    • A/B тестирование стратегий")
print("    • Корректировка подходов based on результатов")
```

🎯 ПРИОРИТЕТЫ И ROADMAP:

- ```
=====
```
- ```
=====
```
1. 🚀 ВЫСОКИЙ ПРИОРИТЕТ (0-3 месяца):
 - Кластер 2: Срочные меры по удержанию (80% оттока!)
 - Кластер 1: Программа возврата клиентов
 2. 📈 СРЕДНИЙ ПРИОРИТЕТ (3-6 месяцев):
 - Кластер 3: Вовлечение в коммуникацию
 - Кластер 0: Развитие премиум-сервиса
 3. 🎯 КЛЮЧЕВЫЕ МЕТРИКИ УСПЕХА:
 - Общее снижение оттока на 15% за 6 месяцев
 - Увеличение LTV на 10%
 - Рост согласия на коммуникацию на 20%
 4. 📊 МОНИТОРИНГ И АНАЛИТИКА:
 - Ежемесячный tracking кластеров
 - A/B тестирование стратегий
 - Корректировка подходов based on результатов

ОТЛИЧНО! 👍

Молодец, в разделе выбора сегмента и разработки маркетинговых предложений по увеличению покупательской активности для него я не вижу никаких проблем. Проведен и аналитический и графический анализ, в выводе находится саммари по всем, наиболее значимым поинтам анализа. Что же касается рекомендаций, то конечно здесь написать можно много ... твой анализ (и графики для признаков) дают массу полезной информации о характеристиках групп покупателей. Написать можно много, но для начала достаточно и тех моментов, которые упоминаешь здесь ты. Молодец!

1. Модель предсказания

- Лучшая модель: SVC с ROC-AUC = 0.90

Топ-5 важных признаков:

- Согласие на коммуникацию (0.0689)
- Товары в корзине (0.0511)
- Время на сайте (0.0357)
- Акционные покупки (0.0326)
- Выручка (0.0302)

2. Сегментация клиентов Выявлено 4 четких кластера с разным поведением:

- Кластер 0 (36%): Лояльные активные клиенты (17% оттока)
- Кластер 1 (29%): Рисковые среднеактивные (51% оттока)
- Кластер 2 (13%): Акционные нелояльные (80% оттока) ← Критическая группа
- Кластер 3 (22%): Пассивные отказчики (31% оттока)

3. Главные инсайты

- Согласие на коммуникацию - самый важный фактор активности
- Клиенты с товарами в корзине требуют особого внимания
- "Акционные" клиенты показывают высокую зависимость от акций (94% покупок по акциям)
- Прибыльность практически не отличается между кластерами

Рекомендации для бизнеса 🎯 Срочные меры для Кластера 2

- Внедрить персонализированную программу лояльности
- Настроить систему напоминаний о корзине
- Разработать постепенный уход от акционной зависимости

📈 **Стратегические инициативы**

- Увеличить долю клиентов, согласных на коммуникацию
- Внедрить сегментированный маркетинг
- Создать систему премиального обслуживания

Ожидаемые результаты

- Снижение общего оттока на 15-20% за 6 месяцев
- Увеличение среднего чека на 10-15%
- Рост LTV на 10-12%
- Увеличение конверсии корзины на 15-20%

План реализации

- Месяц 1-2: Внедрение для Кластера 2 (критический)
- Месяц 3-4: Расширение на Кластер 1 (средний риск)
- Месяц 5-6: Оптимизация для всех сегментов

Заключение

- Проект успешно достиг поставленных целей:
- Построена эффективная модель предсказания
- Выявлены ключевые сегменты клиентов
- Разработаны персонализированные стратегии
- Определены измеримые KPI

ОТЛИЧНО! 👍

Артемию, спасибо за качественный и детальный финальный вывод, равно как и за ход проекта - действительно содержащий важные рекомендации бизнесу и подводящий топ-менеджмент Заказчика к идеям по "донастройке" бизнеса.

Молодец, выводы проекта выше всяческих похвал. Скурпулёзно, детально, с собственными мыслями и рассуждениями, вдобавок ещё и оформлено ОТЛИЧНО!

Результат ревью:

Я отмечаю адекватный уровень твоей программистской подготовки. Ты используешь продвинутые концепции языка Python, библиотеки pandas и одной из основных библиотек машинного обучения scikit-learn. Всё это в целом упрощает твой код и одновременно показывает твой уровень владения инструментами анализа данных и machine learning.

С комментированием никаких проблем нет. Твоих объяснений достаточно для понимания твоими коллегами хода мыслей. Ссылки из брифа проекта в нужных местах также помогают следовать за развитием событий и дают полную информацию касательно того, что будет происходить в том или ином разделе проекта.

Мне было просто проверять твой проект, потому что он чётко структурирован, с использованием оглавления.

Зелёным цветом я отметил и прокомментировал удачные и элегантные решения, на которые ты можешь опираться в будущих проектах.

Жёлтым цветом я выделил то, что в следующий раз можно сделать по-другому. Ты можешь учесть эти комментарии при выполнении будущих заданий или доработать проект сейчас.

Однако есть принципиальные моменты, которые не позволяют мне принять твой проект в его нынешнем состоянии:

Необходимо исправить ❌

1. (45) Кодирование нужно делать только с применением обучаемых трансформеров. Для кодирования целевого признака нужно использовать LabelEncoder: <https://scikit-learn.org/stable/modules/generated/sklearn.preprocessing.LabelEncoder.html> (<https://scikit-learn.org/stable/modules/generated/sklearn.preprocessing.LabelEncoder.html>).
2. (50) Ряд признаков в наших данных имеет дискретную природу (например это: 'Маркет_актив_тек_мес', 'Средний_просмотр_категорий_за_визит', ... (не привожу все названия из данных)). Для графического анализа дискретных признаков нужен тип графиков, который на каждое уникальное значение признака строит свою корзину. Например, можно использовать **sns.countplot** или **.plot(kind='bar')**. Ты же для визуализации их используешь гистограмму.

3. (55) Из финальной версии датафрейма "неактивные пользователи" удалены?
Если нет, то это следует сделать.
4. (56) Нормально ли распределены наши признаки? Если нет, то каким методом надо считать коэффициенты корреляции?
5. (64) Не выполнено требование ТЗ в части использования энкодеров и/или скейлеров.
6. (64) Не вижу обоснования выбора и использования конкретной метрики.
7. (64) А зачем нам в проекте нужна была тестовая выборка? Почему она в этом разделе нигде не появилась и нигде не была использована?

Эти моменты требуют твоего внимания. Комментарии по ним помечены красным цветом.

Подводя итог: хорошая работа на самом-то деле, молодец! Есть несколько моментов к исправлению, но уверен, что они не вызовут у тебя сложностей. После доработки обязательно проверь комментарии и выводы. Их нужно будет скорректировать. Жду доработанную версию проекта на повторное ревью.